

MASTER'S THESIS 2026

Level Design Guidelines Applied to Procedural Open-World Games

How to implement level design in procedural open-world games
following the intended game experience

Raúl Martín



UNIVERSITY OF
GOTHENBURG



CHALMERS
UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering
UNIVERSITY OF GOTHENBURG
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026

Level Design Guidelines Applied to Procedural Open-World Games
How to implement level design in procedural open-world games following the intended game experience
Raúl Martín

© Raúl Martín, 2026.

Supervisor: Michael Heron, CSE
Examiner: Aris Alissandrakis, CSE

Master's Thesis 2026
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg
SE-412 96 Gothenburg
Telephone +46 31 772 1000

Typeset in L^AT_EX
Gothenburg, Sweden 2026

Level Design Guidelines Applied to Procedural Open-World Games
How to implement level design in procedural open-world games following the intended game experience

Raúl Martín

Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

Abstract

Procedural Content Generation (PCG) is widely used in open-world games to address issues of scale and content creation. However, it is often driven primarily by technical considerations rather than experiential intent. This thesis investigates how level design principles and practices can be used to drive procedural generation algorithms in open-world games while being aligned with an intended game experience. The concepts of game experience, open-world games, level design, and procedural content generation, are used as the four foundational pillars of this work. This thesis explores the interconnections between level design and the rest of the pillars, and synthesizes the findings into a set of design guidelines divided into 3 categories: game design, guidance, and spaces. These guidelines are then used in a practical prototype that explores how experiential design principles can be translated into game systems through different PCG techniques. The research adopts an exploratory approach grounded in Research Through Design, positioning implementation as a means of probing feasibility rather than empirical validation. While the prototypes are not evaluated through player studies, they exemplify how level design theoretical tools can be used to inform procedural systems that follow player experience considerations. This work contributes a conceptual framework and practical grounding for future investigations into experience-driven PCG in open-world games, highlighting both the potential and the importance of the presence of level design when implementing procedural generation algorithms.

Keywords: Level Design, PCG, Open Worlds, Gaming Experience, MDA, Guidelines

Acknowledgements

I want to thank my supervisor for supporting me and giving me advice to approach this monumental task.

I specially want to thank my family and my close friends with which I could share my excitement and frustration about the topics explored in this work.

Raúl Martín, Gothenburg, 2026-05-15

«Twill be a cold, dark, and very gentle
place. And one day, it will make someone
a goodly home.»

The Painter, *Dark Souls III* (2016)

Contents

1	Introduction	2
1.1	Notation	3
2	Background	4
2.1	Gaming Experiences	4
2.2	Open-World Games	6
2.3	Level Design	7
2.4	Procedural Content Generation	8
2.4.1	Benefits and Drawbacks	9
2.4.2	Historical Context	9
2.4.3	Classification	10
2.4.4	Methods	11
2.4.5	My Contribution	13
2.5	Guidelines VS Design Patterns	14
2.6	Why this research matters	15
2.7	Level Design and PCG Integration	16
3	Theory	18
3.1	Research Approach	19
3.2	Player Experience	19
3.3	Procedural Content Generation	20
3.4	Guidelines	21
3.5	Prototypes	21
4	Methods	23
4.1	The Research Process	23
4.2	Structure of the Guidelines	25
4.3	Construction and Purpose of the Prototypes	28
5	Results: Theoretical Artefact	29
5.1	How the Guidelines Were Conceived	29
5.2	Overview of the Guidelines	30
5.3	How Level Design Connects to the Rest of the Pillars	32
5.3.1	Level Design in Open Worlds	32
5.3.2	Level Design with Procedural Worlds	33

6	Results: Technical Artefact	35
6.1	Open-world adventure: Floating Islands	35
6.2	World Map: Guidelines to Generation Rules	40
7	Conclusion	46
7.1	Discussion	46
7.1.1	Takeaways of the Guidelines	46
7.1.2	Takeaways of the Prototypes	48
7.1.3	Validation	49
7.1.4	Meaning in Procedurally-Generated Worlds	50
7.2	Future Work	51
7.3	Closing Remarks	52
	Acronyms	53
	Ludography	53
	Bibliography	54
A	Obsidian Diagrams	I
A.1	Level Design Guidelines	IV
A.1.1	Guidelines: Game Design	IV
A.1.2	Guidelines: Guidance	VII
A.1.3	Guidelines: Spaces	IX
B	Example Vistas from the Second Artefact	X
C	Level Design Sources Outline	XII

1

Introduction

PCG is a highly researched topic in the field of computer graphics and in the industry of video games. It is often framed in a very technical perspective, especially in the terrain generation field, where its research focuses on acquiring high-performance and realistic-looking results. However, in the context of video games, level design plays a very important role in the game experience. This feature is usually disregarded by terrain generation algorithms, which are frequently used in open-world games. The purpose of this thesis was to run deep research to understand both level design and PCG, and the different ways they could be integrated to form an engaging game experience.

The research in this thesis can be summarised with the following question. ‘How can we implement level design in procedural open-world games following the intended game experience?’ From this research question we can extract four pillars to explore, each with several related questions.

- **Game Experiences.** How can we define what makes a game experience? How can we measure how engaging and interesting an experience is?
- **Open-World Games.** What games are we talking about? What features do they share? What games are excluded from this research and why?
- **Procedural Content Generation.** What is PCG and when is it okay to use it? What are its benefits and limitations? What are the different methods we can use to integrate it with level design?
- **Level Design.** Why is level design important? What effect does it have in the game experience? What are the different conceptual design tools professionals use when designing their game levels?

These smaller questions that arise from the research question of this thesis helped guide the research process, since by finding answers to these we can get a more clear picture of what it is this thesis tries to achieve.

The main objective of this thesis was to explore how level design interacts with the rest of pillars. The goal consisted of understanding how level design can affect game experiences, which patterns and practices are applicable in open-world games, and how to implement these by using procedural generation. However, due to the fragmented nature of the literature in level design, the work of this thesis focused on building a foundation for its future research.

With this, the presence of the rest of the pillars got diminished, but the connection they share with level design is interesting. That is why this connection is still explored in the results and discussion of this thesis.

The specific work of this thesis consisted of answering all the previous questions via a deep research, and then synthesizing and summarizing all the findings in such a way that the information is more digestible. With that, two artefacts were constructed: one theoretical and one practical. The theoretical artefact is the ‘Level Design Guidelines’ [1]. This artefact collects all the findings from the research and organises them in conceptual categories and sections, providing a learnable and applicable resource. The practical artefact aimed to build a prototype that could display these guidelines by implementing them using PCG algorithms. Here, two different prototypes were constructed. The first one aimed to be a playable open-world game where these guidelines could be identified within the systems. However, this prototype started with too much work put into the implementation of game systems instead of focusing on the implementation of the guidelines, and it ended up being discarded. It still provided interesting insights on the complexity of this translation of level design rules into procedural algorithms, so this is explored in the results section. The second prototype pivots the objective of the first one and instead of aiming for a playable demo where these guidelines can be identified, it just creates a dummy environment where these rules can be matched one to one with specific algorithm rules.

The scope of this thesis reflects an intentional breadth. Rather than addressing a narrowly defined problem, this work aims to construct a foundation for level design while it explores several considerations such as their implementation through procedural methods, or their influence on game experiences. While this breadth enables the creation of this artefact, the scope necessarily constrains the depth with which each individual guideline can be developed and evaluated. The result of this work prioritises integrative understanding of the four pillars over isolated solutions to specific scenarios. This frames the thesis as an exploratory mapping that serves as a basis for further research.

1.1 Notation

Due to the amount of concepts present in this work, a specific notation has been used to ease the process of identifying them in the text and improve readability. Additionally, a list of acronyms [p. 53] can be found at the end of the document.

- **Soft emphasis:** Italics uncapitalized e.g. “This is some *important* concept”.
- **Strong emphasis:** Bold uncapitalized e.g. “This is **crucial** for...”.
- **Level Design Key Concept:** Italics capitalized e.g. *Landmarks* or *Valves*.
- **Guideline Name:** Bold capitalized e.g. **Goals** or **Meaning**.

2

Background

This chapter provides context on the work of this thesis for each pillar, and summarises the literature review.

- **Game Experiences.** A formal definition is provided to understand the importance and implementation of experiences in games.
- **Open Worlds.** This section defines what game genres in particular are targeted by this thesis, and the common characteristics they share.
- **Level Design.** Apart from defining what it consists of, this section also covers some historical context and explains the relevance of the role in the industry and in games.
- **Procedural Generation.** This section explains what procedural generation is, what are its benefits and drawbacks, and how its presence in games evolved over time. Additionally, it lists several classification parameters and methods that can be accounted for in the work of this thesis.

Additionally, this chapter explains why I decided to implement guidelines instead of other formats of knowledge such as design patterns, and it states why the work of this thesis is important. Finally, this chapter explores the current existing literature for the integration of level design with procedural content generation.

2.1 Gaming Experiences

Hughes [2] explains in his thesis how there are two different types of *experiences* to consider: overarching and contextually situated. On the one hand, he defines the *overarching* open-world experience as the freedom the player has to self-pace and auto-impose goals. On the other hand, the *contextually-situated* experiences are defined as the moment-to-moment events the player goes through in a playthrough. The overarching experience can also be seen as the collection of all the contextually-situated experiences. The framework proposed in his findings is based on motivation and goals, which are deeply looked at through several studies, but the results are inconclusive. However, good insights can be extrapolated from this research. Hughes treats both motivation and goals as separate things, where **gameplay** is driven not only by player intent (*motivation*), but *goals* also influence their actions. He does mention how one has a strong effect over the other.

However, this thesis challenges Hughes' and proposes that motivation comes directly from the desire to achieve goals, and the distinction comes from whether these goals are *self-imposed* or presented *by the game*. This implies that the motivation behind any action the player takes comes from their **desires**. The different types of desires are explained in Figure 2.1.

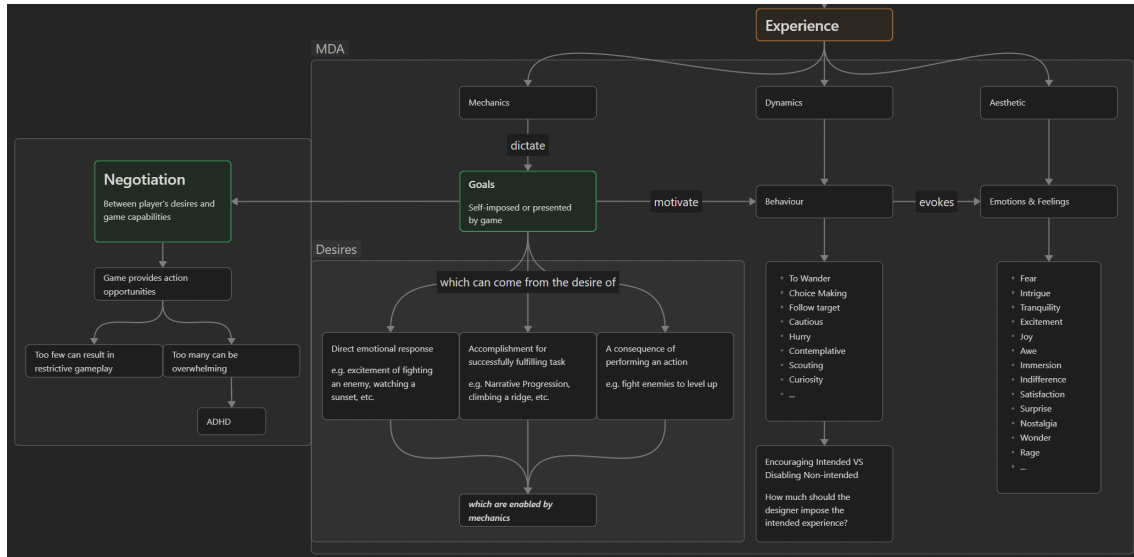


Figure 2.1: Experience concept diagram designed in *Obsidian*. It exemplifies how the MDA framework is structured, and how goals can be born from the player desires.

In this graph we can see how the definition of experience we use goes as follows: an experience consists of the desire, the behaviour, and the emotional response a player has in the game. This mirrors closely the MDA framework [3], one of the most relevant pieces of research in game design, where a game can be analysed from the perspective of its mechanics, dynamics, and aesthetics, and how one affects the others. In our case: **goals** (enabled by mechanics) *motivate* a **behaviour** (dynamics), that later evokes **emotions** (aesthetics).

In his work, Hughes [2] explains how a gameplay experience consists of a constant ‘negotiation’ between the player and the game, where the player has a desire to perform certain actions, and the game responds with its available systems or goals. This is particularly important in open-world games, where the freedom to explore the world relies on a much richer and more profound aspect of *Goal Layering*. In this context, level design is the one in charge of distributing said goals to construct the intended experience. Here is where the connection between open-world games and level design lies. This research ignores what type of game systems enable the different types of goals that can be created and instead focuses on exploring how the distribution of these goals is presented to the player through level design.

2.2 Open-World Games

Open-world refers to a genre of video games where players can freely explore a large, usually continuous game space and decide for themselves what to do and when to do it. This is what we call *Player Agency*, where, instead of being pushed along a strictly linear sequence of levels or missions, the player can roam the world, ignore or postpone main objectives, pursue side activities, and often shape their own experience through **choice** of activities, playstyle, and priorities. The core idea is that the player can self-pace their gaming sessions. The narrative, the game progression, and the challenge are usually structured around this principle to support this player agency rather than dictate a fixed route.

To support this sense of freedom and exploration, open-world games tend to share several common mechanics and design elements. These elements can appear in many genres, but in open-world design they are usually arranged together [4].

- **Exploration and traversal:** Large maps, subdivided into multiple biomes or districts, and various traversal methods (e.g. on foot, vehicles, mounts, climbing, gliding, fast travel, etc.).
- **Non-linear objectives:** Different orders of importance for the goals: main story quests, side quests, activities, collectibles, and sometimes dynamic events. These can usually be approached in any order or even ignored by the player.
- **Progression and gameplay variance:** Experience systems, different types of gear, skill trees, and upgrades let players specialize (combat, stealth, crafting, exploration, etc.) and approach challenges in varied ways. Depending on what genre the game is there might be more or less freedom in this aspect. One such example is the RPG *Elden Ring (2022)*, where the build you choose drastically changes the way you approach combat, whereas in *Zelda BOTW (2017)* you can just upgrade your backpack size and your health and stamina.
- **Discovery-driven content:** Hidden locations, collectibles, environmental storytelling, and optional quests or encounters reward player *curiosity*.
- **Sandbox interactions:** These games often include many different tools which support multiple solutions to problems rather than a single correct approach. Some examples include physics, crafting, or building.
- **Systemic world simulation:** Some open-world games also include dynamic systems, such as day-night cycles, weather, NPC schedules, and ecosystem simulation (e.g. wildlife) to help the world feel alive beyond scripted sequences.

The open-world genre describes the structure of the game where these listed characteristics are commonly present. However, it does not dictate its core activity or theme. Because of this, it acts as a layer that can sit on top of almost any primary genre. It is typically used as a qualifier (open-world RPG, open-world survival horror, open-world action-adventure...) rather than as a fully self-contained label, because the underlying playstyle is still defined by the primary genre.

This thesis aims to provide a set of level design guidelines that apply in any of these genres. This means that games like *Minecraft*, *Assassin's Creed*, *Firewatch*, *The Evil Within*, *Satisfactory*, *Elden Ring*, *Subnautica*, and *Goat Simulator* could benefit from some of the insights of this thesis, even when they are so different from each other. Not all of these games would benefit from an in-game procedurally generated world, since some of them have a special design considerations, such as a strong narrative, but even then PCG could be used in the development phase as a tool to accelerate specific parts of the process.

It is important to remark that some genres are disregarded in this thesis, since their level design needs are quite distant from the genres listed before. These include genres like platforming or racing, which rely on highly controlled level layouts and fixed paths. These can coexist with the open-world genre, but the focus of the level design research in this thesis is not in how to space out platforms and make interesting circuits to drive. Level design in this thesis refers to the overarching structures and the cognitive influence different rules have over the player's intent.

2.3 Level Design

The foundation of level design consists of **guiding** players through **space** using layouts and visual cues to support gameplay, pacing, and narrative [5]. It can be defined as the process of planning, structuring, and implementing the spaces in which a game takes place. A level designer's role is to assemble the core gameplay elements in such a way that it encourages the intended player experience [6].

In the contemporary game development industry, level design is a crucial role due to its interdisciplinary nature [7]. It holds a deep connection with every other specialization. Game design defines the core mechanics, systems, and progression (along with narrative design). Game art first delivers thematic context and provides the building blocks of the scenery, and then it gets constrained by the layouts provided. Technical roles need to discuss with level designers the capabilities and limitations of the systems being developed. In this matter, the level designer often operates as an **architect**, integrating all these elements together within coherent, navigable, and readable spaces [8].

Historically, level design emerged as a more informal practice tightly related to game design. It was usually carried out by programmers, who manually built stages using simple tools or even raw code [9]. The earliest games that featured modern properties associated with level design include *Super Mario Bros* (1985), *The Legend of Zelda* (1986), and *Metroid* (1986) as the most famous examples. Their rules were strongly dictated by hardware limitations and simple rule sets, but still they laid the foundation for principles such as difficulty curves, readability, spatial progression, exploration, visual cues, etc. Foundational examples are harder to enumerate, since they define a more blurry outline of the current practices associated with level design. Some of these games include *Colossal Cave Adventure* (1976), *Adventure* (1979), and *Pitfall!* (1982). As games increased in complexity and production values, level design gradually evolved into a dedicated discipline with specialized tools, pipelines,

and methodologies [10]. The result is that level design today is recognized as a core role that significantly influences both player engagement and the overall experience of a game [11].

The impact and importance of level design has become especially pronounced in open-world games, where the player is granted a high degree of agency and control over pacing and goal selection. There is a strong difference with how level design works in games with a linear nature. Linear games provide more control to the designer in the sense of **Pacing**, **Pathing**, and all the factors that affect **Engagement**. The effect of how much control the designer has over these factors is debatable, since a player deciding what to do usually grants them a more authored experience, but they can also get lost and choose to play in a way that stops being engaging after a while. In open-world games, designers must consider how exploration is motivated and account for different layers of goals at any time. This translates into a different approach, where they have to think about how **Points Of Interest (POIs)** are distributed across the world, and how players are subtly guided without overtly constraining them. Level design in open worlds involves crafting hierarchical structures (*Regions*, *Hubs*, *Landmarks*, etc.) that support orientation and navigation, while also accounting for localized gameplay instances that maintain variety and pacing. The design sometimes must also account for multiple playstyles, emergent interactions between systems, and the need to sustain engagement over extended play sessions. Hence, while level design in open worlds has a distinct nature, its quality is still crucial to provide engaging player experiences. For this reason, procedurally generated worlds should be constructed upon level design principles.

Note that this thesis focuses on level design as in the context of world structure and integrating mechanics to convey the intended gaming experience. Level design can sometimes be used as an *umbrella* term to also refer to level art, which informs the placement of assets, lights, and other level elements to provide a specific aesthetic. Level designers should work hand in hand with level artists, but their roles should not be confused. Visual dressing and stylistic choices are acknowledged as influential in terms of player guidance, but fall outside the main scope of this work. This allows for a more focused investigation into the principles and practices that dictate level design, particularly within the context of open-world games.

2.4 Procedural Content Generation

Procedural Content Generation (PCG) consists of the usage of *algorithms* and *randomness* to automatically create game content [12]. This means that these game elements are generated by code and Random Number Generators (RNGs), as opposed to handcrafted content. Its purpose is to produce content that is both varied and scalable in a dynamic way, reducing the production cost. Some examples of content that can be generated procedurally include level layouts, quests, loot systems, character appearances, and terrain, being the latter the most heavily researched topic in the literature. Some examples of PCG methods typically used in video games include Perlin Noise, Wave Function Collapse, Grammars, and Random Walk, among many others.

2.4.1 Benefits and Drawbacks

The main benefit PCG provides is **replayability**, or infinite play. This is because with procedural levels or scenarios we can have an experientially infinite set of content or outcomes [13]. This is the case of *Noita (2019)*, a roguelike video game where a system of caves and the selection of items and enemies found in it are generated procedurally. Another benefit is **adaptability**, where the *uncertainty* of events leads to a more dynamic playstyle in which players have to constantly react to new scenarios [14]. Furthermore, some game elements can be tweaked by the PCG algorithms to adjust pacing and progression dynamically so that levels get more intricate and difficult over time, while at the same time they get balanced with the skill the player presents. **Unique experiences** can also be experimented, this referring to one player having an experience that no other player has had before. This is the case of exploration in *Minecraft (2011)*, where the idea of discovering never-before-seen lands can provide a very specific sensation that is impossible to recreate with handcrafted content [15].

However, PCG also presents some issues mainly related to the **complexity** of the implementation of algorithms and the ‘interestingness’ of the results. This complexity can be born from specific game needs, such as the requirement of existence of a *critical path* i.e. a valid path for the player to achieve its goals. For some algorithms it can be really complex to determine if this exists, even more if the game requires to only have a single solution, such as many puzzle games. Measuring ‘**interestingness**’ is another problematic task due to its subjective nature. One example of a missing interesting game element when using PCG is *noteworthy gameplay instances*. These are incredibly difficult to provide since they are usually meticulously crafted by hand and are most likely to be tied to narrative e.g. going back to your home town after defeating the main antagonist only to realize it has been invaded by goblins. The creation and relevance of this **Meaning** is explored both in the guidelines and in the discussion [Section 7.1.4].

2.4.2 Historical Context

In recent decades, PCG has undergone a significant evolution within the field of video game development. Early uses of PCG focused primarily on simple randomisation to extend replayability or reduce memory usage (*.kkrieger (2004)*), such as generating basic dungeon layouts or random enemy placements. Today, however, the most advanced implementations of PCG are deeply integrated into the core of games. One such example is terrain generation, which has reached a high level of precision and efficiency, employing noise functions, layered heightmaps, and sophisticated erosion simulations to produce believable landscapes at scale. Another specific example that is gaining popularity lately is voxel engines. These enable the creation of highly detailed, fully mutable game worlds, allowing both developers and players to manipulate the environment in real time, as is the case of *Hytale (EA 2026)*. Voxel renderers have existed for a long time, one of the earlier examples being *Comanche Maximum Overkill (1992)*. They have later been widely popularised with *Minecraft (2011)*. Recently, with the advancements of technology and research,

people are achieving finer results with higher resolution while still conserving the mutable property of the worlds. Such is the case of *Teardown (2022)*, where PCG can be applied to generate dynamic environments and highly interactable worlds.

Nowadays, PCG is also used in game development in the process of creating finite worlds by automatically distributing assets, ‘painting’ large sceneries with trees, houses, and other game elements. PCG techniques also extend to the automatic creation of game assets, such as NPCs, items stats, buildings, and even entire cities with level layouts. Using this technology as a creation tool can produce a variety of visually or semantically coherent results by combining modular elements of hand-crafted content. These include clothing, body parts, and animations for characters; different architectural elements for buildings; and pre-designed rooms as building blocks for larger level design contexts. PCG is no longer limited to simple randomization but is instead a powerful tool that supports game content creation and reduces the manual workload on development teams.

2.4.3 Classification

This section presents a list of properties with which we can classify procedural generation algorithms. Listing these properties in this context is useful because it can help identify the specific characteristics we desire in our algorithms [16].

- **Input Required.** What type of input the algorithm requires to run. It might require no input, user/designer input, or training data.
- **Data Representation.** Output data can be represented in many different ways: as textures, as splines, as vectors or graphs, as voxels, as rules, etc.
- **Controllability.** This defines how much control we have to alter the results of the generator. We can have no controllability (only seed-based), suggested control (e.g. parameters), or precise control (e.g. sketch-based). This category can be split up depending on if we are talking about user or designer controllability. Furthermore, we can also differentiate if this control is applied before running the algorithm or during its execution.
- **Evaluation of Success.** A successful result can sometimes be identified automatically, but some other times we require user-evaluated verification. Automatic evaluation of success is especially useful in algorithms that require parameter tuning.
- **Complexity.** We can measure complexity based on the number of features and layers our generator has. We can also identify characteristics such as testability or ease for implementing intricate rules.
- **Computational Cost.** We can measure memory usage or execution time, and evaluate which one is a stricter constraint for our application. We can have offline or runtime algorithms, which particularly influences the importance of execution time.

- **Determinism.** Depending on how we construct our algorithms, the implementation of a seed can provide deterministic results. If we do not require this functionality we can be more relaxed on our constraints.
- **Reusability.** An algorithm that can be tuned to many different purposes is considered reusable. This can be the case for a flexible world generator, where you can alter the frequency and intensity of specific features, such as mountains, water bodies, and biomes.

Some other characteristics are more closely related to the paradigm or purpose of the algorithm. This can be the case of chunk-ability in world generation, where the generation can be split up in small plots of land instead of all at once, or traverse-ability, which dictates if there is a path for the player to walk from place to place.

2.4.4 Methods

This is a specific type of categorisation where we distinguish algorithms depending on their paradigm [12], [17]–[19]. Bear in mind the following categories are not mutually exclusive, but algorithms might belong to more than one of these. There are four major groups, which can be further divided into more specific categories:

- **Stochastic.** These are algorithms that try to imitate natural randomness. They use rules and parameters, but lack precise control over resulting features.
 - *Fractal.* These refer to algorithms whose rules are defined recursively. Some examples include layering noise textures like Perlin Noise, or implementing the Random Mid-Point Displacement algorithm.
 - *Grammars.* These are a specific type of fractal algorithms which define elements and rules like grammars do. These then converge into terminal symbols that represent the result of a generation.
 - *Tiling.* These refer to algorithms that subdivide the space. They can apply different rules at different depths. An example algorithm is Binary Space Partitioning (BSP).
 - *Parametrization.* These algorithms define parameters that can be used to adjust the behaviour of the generator. They are most commonly applied in Distribution Sampling, to define frequencies and probabilities of elements being selected.
- **Constraint-based.** These are algorithms whose generation is determined by a strict set of rules that has to be solved for in order to generate valid states. One specific example is Wave Function Collapse (WFC), which defines a set of tiles and rules on how they can be combined together.
 - *Constructionist.* These refer to randomly stitching modules to generate a variety of possible sequences. They are typical in dungeon generators for *roguelike* games. These modules may be generated by hand or by another PCG algorithm.

- *Cellular Automata*. This is a specific category that works within a grid space and has rules to define what the next step of generation is based on its current state. The most famous rule set is the one from Conway's *Game of Life* (1970). This category could also belong to **Simulation**.
- **Simulation**. Algorithms in this category try to recreate natural world occurrences. They have some initial parameters and state, which is then modified over time. Usually, the designer cannot intervene in the generation process until the final result is provided. They also tend to be slow and lack precise control over features.
 - *Geomorphological*. These simulate terrain generation occurrences, like river erosion or mountain ranges creation with tectonic plates.
 - *Ecosystem*. These refer to the evolution of the fauna or flora, or even the effect they have over the environment. They can dictate which populations of entities persist, or how the environment is modified by their presence, by changing the state of the soil or tracing paths. This category can also include human behaviour, where they can drastically modify their surroundings by creating towns and roads.
 - *Agent-based*. These refer to effectors that do not base their behaviour on geomorphological or ecosystem events, but they are straightforward modifiers that can be applied randomly. Some example effectors include rising, flattening, or smoothing surfaces, texturing effectors, or river effectors that connect points on two distinct bodies of water.
- **Learning**. These algorithms process provided data, usually in an iterative way, to replicate their intrinsic patterns. The quality of the provided data here is crucial to have a successful generation.
 - *Example-based*. Here usually real data is used as atomic modules that can be stitched together with other generation methods. The properties of these examples are used to replicate their looks while combining them all. This is similar to the constructionist approach.
 - *Search-based*. These algorithms are based on a fitness function they use to evaluate results and iterate on by modifying them slightly until a local minimum is found. The fitness function has to be defined around the properties of the data we want to replicate. This is typical of Evolutionary Algorithms, where different chromosomes define possible solutions, and they compete with each other until the most fit survives.
 - *Inverse Procedural Generation*. Extraction of parameters from either real data or a provided handcrafted sample (e.g. creating a flora painting brush given the distribution of plants and trees in a plot of land).
- **Sketch-based**. This category refers to the PCG algorithms that act as tools designers can utilize to populate levels. They are the kind of generators with most precise control over the generated features. The name comes from the specific tools that can receive drawings as input to use as templates.

- *2D*. These can be textures that define water bodies or mountain ranges in the world, or probability maps to spawn enemies or Points Of Interest.
- *3D*. Although these are more complex to work with, they provide flexibility to the user to define characteristics of their games in a 3D space. Some common tools are splines or 3D effectors that specify how game elements should be spatially generated in specific parts of the level.

Some specific examples of procedural generation techniques include: layering noise, graph generation and traversal, irregular grids, and usage of mathematical distributions. Specific named algorithms include: WFC, Marching Cubes, Random Walk, Distribution Sampling, BSP, Delaunay Triangulation, Markov Chains, etc.

2.4.5 My Contribution

This thesis investigates how level design rules can be embedded directly into a game through the usage of different types of procedural generators, with a specific focus on level layouts for open-world games. Rather than treating PCG as a purely technical system that generates arbitrary content, the focus is on encoding design knowledge (e.g. constraints on navigability, pacing, difficulty curves, and visual language) into the generative process itself. By integrating level design rules, generators can produce content that is not only structurally valid but also aligned with the **intended player experience**. This involves translating abstract design principles from natural language into formal constraints and heuristics that can guide generation in all these ambits. The overarching goal of the thesis is to bridge the gap between the cognitive and often intuitive practice of level design, and the systematic nature of procedural algorithms.

A conceptual distinction to be made in this thesis is between what can be referred to as technically procedural and meaningfully procedural content. On the one hand, **technically procedural** content is any content generated by algorithms using some form of randomness or rule-based logic. For example, assigning enemies random amounts of damage values or generating loot with randomised statistics falls under this category. Such uses of randomness often have limited impact on the players qualitative experience of the game. While they may increase variability, they will not be fundamentally changing how the game is played or perceived. This was the usual case in early instances of PCG in video games, but it still has a strong presence in the modern industry. On the other hand, **meaningfully procedural** content refers to content whose generation has a deliberate, perceivable, and coherent effect on the overall experience or interpretation of the game. These are usually more complex to implement and affect aspects of a game such as encouraging exploration, supporting game pacing, or reinforcing particular design goals. With this distinction, we can set the research background focus in the latter type of Procedural Content Generation. The integration of **open-world level design** principles into these **generators** leads to following the intended design **experience** from a programmatic perspective, covering all four pillars of research.

2.5 Guidelines VS Design Patterns

In game development research, *design patterns* have been an influential tool for describing and analysing recurring solutions to common design problems [20]. These patterns are valuable because they offer a shared vocabulary for developers and researchers, making it easier to identify and compare design elements across different games. However, despite their usefulness, design patterns tend to remain relatively modular units of knowledge. Each pattern typically encapsulates a specific *problem-solution* pair, which can lead to a fragmented understanding of complex domains such as level design [21], where many interdependent factors must be considered simultaneously.

Design patterns are frequently compiled into large catalogues, which can become collections of loosely related entries rather than a tightly integrated body of knowledge. For a topic as intricate and layered as level design, this can make it difficult for designers and researchers to grasp the larger conceptual structure. One particular example of this is observed in the industry professional Peter Field’s talk about spatial communication [5]. This talk goes over many different design patterns and tricks that give great insights about what the player experience might be when presented with different visual cues and layouts. However, all these tips feel somewhat disconnected from each other, and it can be overwhelming to try to apply all of them at the same time when designing your own levels. Among these patterns he explains concepts like clear goals, sightlines, several pathing tools, vistas, affordances, and even how to influence the player’s curiosity with artistic features. These are all really useful concepts that ended up integrated in the guidelines, but the contribution of this thesis provides an artefact where these are presented in a more structured and thematically cohesive narration. Here, guidelines do not replace the insights of design patterns, but rather build on them within a more educationally oriented framework.

On a more general note, many professionals in the industry have shared similar contributions before [22]–[24], providing their own practices and insights of their approach and their experience in many different contexts, from indie games to AAA industries. These resources have been essential to build the knowledge base for the level design pillar in this thesis, but still, most of them lacked cohesion in all the concepts presented. This has been a running issue throughout the literature, where design principles, practices, and considerations were presented alike, as if they provided the same kind of tools to the reader. They all also state how these insights are never sufficient to do proper level design, and that designers require a kind of tacit knowledge that is only acquired through practice. In addition to all of this, none of them explicitly mentioned the close relation level design has with game design itself, and how many decisions depend on a close discussion with the development team.

The work of this thesis aspires to deliver the **foundation** of a holistic and practically actionable body of knowledge. In the theoretical artefact, guidelines are presented following a textbook-like structure. Each guideline shows a solid structure that progressively builds an understanding of level design. These are also separated from

designer considerations and practices, which belong to additional sections. Furthermore, this text provides a practical pipeline that explains in which order to take guidelines in consideration throughout the lifetime of a project. Additionally, unlike individual patterns, each guideline establishes its relationship with others, uses metrics, and outlines real industry examples to provide a complete context on what these rules are, when they should be applied, and why.

This way of structuring all the knowledge allows the material to support both learning and practice. Newcomers can use it as an **educational resource** that supports their learning process, while experienced designers can use it as a **toolkit** that they can check to refresh key concepts. The aim is not just to enumerate known solutions, but to offer a structured approach for reasoning about levels, from high-level design decisions down to moment-to-moment player experiences.

2.6 Why this research matters

First and foremost, the importance of this thesis relies on the relevance of the research question itself. We want to understand how level design can influence the game experience in an open-world game, and then apply it algorithmically through PCG. There are several links here: how game experiences and open worlds connect, how level design can influence these experiences, and how to implement level design with procedural generation.

To start off, we can use the definition of an open-world experience provided in Section 2.1. It explains how a experience is constructed upon the MDA framework, and how the mechanics dictate what are the possible goals. These goals can also be born from the desires the players have, and the discussion players have with the game to understand what are possible interactions is called *Goal Negotiation*. Level design is in charge of laying out these goals as opportunities so that the players engage with them. This is specially tricky in open spaces, since designers have to be more lenient on what the intended experience can be due to players choosing what to engage with at any point. On top of that, level design can be understood as the **guidance** of players through **spaces**. Non-linear level design is usually explored from the perspective of closed spaces with branching, and open spaces are left aside as simple empty playgrounds to fill up with scattered points of interest. This thesis suggests we can take the principles of linear level design and extrapolate them to a open space context.

The connection between level design and procedural generation is a fresh topic in research, but there is some evidence of people seeking this connection [14], [25]–[27]. The approach of this thesis relies on formalizing the principles of level design and translating this natural language into algorithmic rules to follow. This thesis focuses on emphasizing the importance of this connection, rather than explaining in depth how this implementation could happen. Still, a practical example is presented in the Results [Chapter 6]. Integrating procedural generation with level design opens up a discussion topic about the interestingness of the results due to a possible lack of *meaning*. This is further explored in the discussion [Section 7.1].

2.7 Level Design and PCG Integration

When thinking about integrating PCG with level design there are two distinct parts. We can integrate it with hand-crafted content or with level design principles. Both are crucial to consider to have successful results, although the most interesting topic is the latter, which is the one integration that lacks more research in the literature and which this thesis tries to further explore.

The integration of PCG with hand-crafted content is straightforward. M. Johnson [14] explains two different approaches: horizontal and vertical integration. In horizontal integration, the hand-crafted content follows a modular nature [28]. Here, each piece of generated content present in the game can be replaced by hand-crafted one. This approach facilitates the implementation of set pieces, which directly fights the struggle of PCG to create *noteworthy gameplay instances*. One example of this type of integration can be a dungeon generator where specific designed rooms provide story beats or carefully crafted experiences. A consideration of this approach is whether designers should blur the edges between both types of content or if they should make sure these set pieces are striking enough and not missable [29]. On the contrary, with vertical integration, instead of having scattered elements of hand-crafted content all around the game, the procedural content is constructed around the former. This means that the game will always present the main story line or provide a set of goals, and then randomness is applied on top of that to add replayability. A real industry example is *Spelunky (2008)*, where a trail of specific actions, enemies, and items can be followed to reveal a new secret final area.

The integration of level design principles is substantially more complex [30]. J. Segree [31] explains different approaches to implement PCG in puzzle games. The approach in this thesis builds on these by formally defining goals, rewards, and metrics to understand how to construct a method of generation that satisfies some common level design principles such as *Pacing* and *Vistas*. Some useful metrics to define **Goals** include difficulty, mechanics involved, time to solve, number of actions required, etc. Although this serves the purpose of automated categorization, it might lack designer intentionality if it is not specifically encoded in the generation. It might be harder to present new mechanics to players in the same way that tutorials are designed, and there might be a lack of cohesion since every task is presented in isolation. We can still apply some techniques proper of puzzle games like **batching**, which consists on combining small pieces of puzzles to create larger ones. This concept is later referred as *Gardens* in the level design guidelines. Other useful techniques are **incremental change** (adding constraints in stages with decreasing impact), or **mechanic checking** (forcing a situation where a mechanic is required so that the player can show their understanding of it).

In procedural generation, the approach of implementation can follow two distinct paradigms. The top-down paradigm explains how we first have an example of what the result is and then construct a generator that produces similar outputs, while the bottom-up paradigm first comes up with the algorithm and then tunes it for the most interesting results. The approach to implement the integration of PCG with level design principles following puzzle design concepts is similar. The concepts

here are named solution-down and puzzle-up, respectively. The first one proposes a solution and then adds constraints around it to justify that solution. The second one generates a list of random constraints and then runs a solver to check if a solution exists. A mixed strategy can also be implemented which first generates a solution and then when adding the constraints it might modify slightly the solution to better fit the intended output. For level design we have to be specifically careful with some considerations, like the existence of a *Critical Path*, multiple solutions, undesirable results, and be aware of spatial constraints and the balance between chaos and lack of variety.

3

Theory

Since the scope of this thesis is quite ambitious and it includes a deep research of many topics, along with the implementation of both theoretical and practical artefacts, several theories are required for different purposes. Some of them are used to frame the research problem, while others are used to explain the structure or content of the knowledge, or to justify the approach of research.

Here is an outline of this chapter, which is divided into 5 sections, each with their respective theories:

1. **Research Approach.** This section explains the nature of the research problem through *Wicked Problems* [32], and how the research approach is based on the *Exploratory Research* framework [33].
2. **Player Experience.** This section briefly explains how the *MDA Framework* works and its importance to construct a game experience. After that, it explains how the guidelines of **Goals** and **Engagement** are based on the *Self-Determination theory* [34].
3. **PCG.** Here, the *Systems Theory* [35] and the *Emergence Theory* [36], explain the nature and relevance of PCG. They also explain how this thesis proposes the integration of PCG with Level Design.
4. **Guidelines.** Theories in this section are used to explain the importance of a shared vocabulary in design practice through the use of the *Sapir-Whorf hypothesis* [37] and *Pattern Languages* [38]. Furthermore, it explains the importance of *Affordances* [39] in the context of Level Design.
5. **Prototypes.** Finally, this section explains the relevance of the implementation of the practical prototypes through the theories of *Research Through Design* [40] and *Constructivism* [41].

3.1 Research Approach

Wicked problems [32] are characterised by ambiguous boundaries, multiple stakeholders with conflicting values, and evolving constraints that change as one attempts to intervene. This type of problems are difficult to define or solve since they are strongly interconnected with the context they are situated in, and they often have a subjective nature. This is the case of our research question, where we are trying to evaluate how to influence player experience (an intrinsically subjective matter) through different tools such as level design and PCG. These experiences are strongly context dependent, since they rely on the demographic they are aiming to influence. Framing the problem as a wicked problem justifies the need for a flexible, iterative research strategy rather than a rigid, hypothesis-testing design.

The flexible approach in this thesis started by exploring the research field of both open-world experiences and level design in a broad sense, and then narrowing it down by finding commonalities and selecting the most interesting and related questions. This helped define a more concise research question. However, since wicked problems are context dependent, we need to clearly define the scope to narrow what type of answer we are looking for. In the context of this thesis, this part of the work consisted of carefully defining which specific kind of open-world game experiences we want to explore, as seen in Section 2.2.

The specific approach of this thesis consisted on an *exploratory research* [33]. These are typically used when the phenomenon under study is not yet well understood, theoretical foundations are incomplete, or prior empirical evidence is scarce. This is the case of level design in the context of open-world games. Rather than testing predefined hypotheses, exploratory studies aim to generate insights and uncover patterns that can inform future, more structured investigations. This approach allows the research to iteratively refine questions and potential solution directions as our understanding deepens, working along the theory of wicked problems. Together, the wicked-problem perspective and the exploratory research approach support the idea that the objective of this study is not merely to test an existing theory, but to construct and refine understanding in a complex and only partially understood problem space.

3.2 Player Experience

As explored in the background (Section 2.1), a game experience can be defined through the **MDA Framework** [3]. This framework is widely referenced in the level design literature, and it explains how a game experience is constituted of three parts: the mechanics or systems that define the rules of the game; the dynamics, which refer to the interactions and the emergent behaviour these mechanics allow the player to have; and the aesthetics, which relate to the subjective experience the player has when interacting with the media. This thesis uses this framework as a base for further researching what role level design plays in game experiences and what tools we can use to influence them.

This thesis states in Section 2.1 how the mechanics of a game serve the purpose to construct **goals** or objectives, and how these relate to the **desires** the player has to perform certain actions. Then, level design is in charge of laying out these goals to create a conversation with the player called **goal negotiation**, where the player comes up with desires and the game responds with its available systems. This knowledge about game experiences is collected and presented as the guidelines of **Goals** and **Engagement**, which explore concepts like *Player Agency*, *Challenge*, **Choices**, and **Meaning**.

All these concepts are strongly tied to the *Self-determination theory* [34]. This is another framework that explains human behaviour through the satisfaction of three basic psychological needs: *autonomy*, *competence*, and *relatedness*. In the context of games, autonomy refers to the players perceived agency and freedom to make meaningful choices; competence relates to the players sense of mastery, challenge, and progression; and relatedness concerns the experience of meaning, coherence, and connection to the game world. This theory is relevant in the sense that when we design a game, we do it so that the player has a specific experience, and to achieve it, they have to follow paths or interact with the game in specific pre-considered ways, even when the target experience is based on *emergent gameplay*. The self-determination theory provides tools we can use to analyse games and to understand how to construct them.

3.3 Procedural Content Generation

Procedural Content Generation in games is commonly understood through the lens of *systems theory* [35], in which content is produced by the interaction of rules, parameters, and constraints rather than direct authored control. From this perspective, a PCG system is a structured set of interdependent components whose behaviour cannot be fully understood by examining individual elements in isolation.

A core concept within this framework is *emergence theory* [36], which in the context of video games can be tied to emergent systems or gameplay. Here, emergent behaviour refers to patterns or structures that arise from the interaction of simple rules, producing outcomes that are not explicitly specified by the designer. In PCG, this means that level layouts and gameplay interactions are generated indirectly, making the resulting content partially unpredictable yet constrained by the design of the rules. Emergent gameplay is also predominant in open worlds, since the scale and non-linearity of the space, along with the player agency, disallow for strict designer control. Instead of creating specific experiences, designers influence the space of possible outcomes by shaping generative rules and constraints. Player experience is therefore designed through guidance, instead of strict pathing sequences.

It is important to remark that here PCG is not treated as an automation of level design, but as a design medium that shifts control from hand-crafted content to the construction of generative systems. Designers can use these systems to produce certain experience opportunities while accepting *variability* and *unpredictability* as inherent properties, which are important factors of the **Engagement** guideline.

3.4 Guidelines

The creation of the guidelines as a theoretical artefact of this thesis is based on the *Pattern Languages* [38] theory, but also strongly inspired by the *Sapir-Whorf hypothesis* [37].

A pattern language is a structured set of named, reusable solutions to recurring problems in a given context. Each pattern labels a problem-solution pair and relates it to other patterns, forming a network of concepts that designers can draw upon. In design practice, such a language functions as a common conceptual toolkit that designers can use to coordinate and communicate complex ideas with each other. In this work, guidelines are based on design patterns, but they take an extra step by adding context on how they can be applied in different situations, providing metrics, and giving real industry examples.

The Sapir-Whorf hypothesis proposes that “*The language that we speak shapes the way we think*” [37]. While this thesis does not formally apply or test this hypothesis, it is used to explain how a level design vocabulary can serve as a toolkit for the designer with which to reason about common design problems and solutions.

A specific theory that is widely common in design, and explicitly stated in level design is *Affordances*. Popularized by Norman’s ‘The Design of Everyday Things’ [39], affordances explain how the appearance of objects should hint on their purpose or way of interaction. In level design, affordances become a primary channel of designer-to-player communication: they shape what players immediately perceive as possible or expected actions in a space (e.g. which surfaces can be climbed, which paths can be traversed, or which objects can be interacted with) without requiring explicit instructions. Affordances can also be transcribed from everyday life interactions to video games, but working against them can result in breaking the player’s immersion. Such can be the case of an ajar door not being interactable.

Having a common set of terms for patterns and affordances in level design does not merely ease communication, but can shape how designers perceive design spaces, recognize problems, and construct solutions.

3.5 Prototypes

An important framework to tackle wicked problems is *Research through Design (RtD)* [40]. Rather than treating design artefacts as mere outcomes, RtD states that the act of designing and implementing artefacts serves as a method of knowledge gathering. This knowledge is created through the iterative process of creation, evaluation, and refinement of artefacts, allowing researchers to explore complex problems that cannot be fully understood through analytical reasoning alone. This is the case of level design, which has an intrinsic tacit knowledge associated. As many professionals in the industry put it, one can only truly learn through practice.

This approach is closely aligned with the *Constructivism* theory [41], which states that knowledge is actively constructed through experience rather than passively discovered. From this perspective, understanding how procedural level design influences player experience requires direct engagement with the design process itself. Implementing a prototype enables the exploration of how theoretical design guidelines, generative rules, and constraints translate into concrete spatial structures and playable scenarios. The prototypes therefore function as tools that support learning and prove the theory behind them.

Together, Research through Design and constructivism justify the implementation of the practical prototypes as an integral component of the research process. The prototypes are not presented as finalized solutions or empirically validated systems, but as exploratory pieces of work that demonstrate how the conceptual design guidelines can be translated into procedural systems. Their implementation simply serves to investigate the feasibility of this translation and to expose technical or conceptual challenges and limitations.

4

Methods

This chapter goes through the processes and design choices applied in the thesis. In particular, it explains what the research approach consisted of, and the reasoning behind it. This chapter also presents the roadmap that this whole project followed, and the outline of how both the theoretical and practical artefacts were conceived.

4.1 The Research Process

The main methodology used in the research process of this thesis is the *scoping review*. Unlike other more traditional methods like *systematic reviews*, the scoping review helps exploring the breadth of available research to summarize scattered knowledge [42]. The nature of open-world games is analysed in this research through many different perspectives, but little resources can be found that combine them to have a more broad knowledge of how it all works together. Some of this different research fronts include open-world experiences, level design, environment art (and its connection with *psychogeography*), mechanics, pacing, difficulty scaling, guidance, and the exploration of what makes a space interesting. The usage of a scoping review helps this research collect many different factors that enable a designer create a compelling open-world experience. This is necessary for this thesis to later design procedural algorithms that use this knowledge in several ways. The scoping review method also helps finding gaps in the research field to later seek an answer for.

The literature review of this thesis on level design is based on many different sources, like articles, academic papers, theses, case studies, developer insights, interviews, and Game Developer Conference (GDC) talks, among others [Appendix C]. This variety in knowledge sources represent why a scoping review is necessary, since there is no unified set of knowledge that can be applied to level design, specially in the context of open-world games. Throughout the literature review we can find how this is true, since every designer talks about their experience, and their design tools do not usually match. It is important to note that specifically for the research of level design and open-world games experiences there are not many academic resources, as they pertain to tacit knowledge and professionals' experiences. This is not necessarily bad, but it explains the less formal and more subjective nature of the findings. This is not the case for PCG, as it is presented from a more technical and practical perspective.

The research process was divided into 3 milestones: the literature research, the field research and design, and the implementation. The first milestone consisted of going through the published material related to the 4 pillars of this thesis. The second milestone focused on relating all this gathered knowledge to real game examples to get some insights on the presence of these concepts. This implied playing games like *Subnautica* (2018), an open-world exploration game with narrative on the side, and *Death Stranding* (2019), a strong narrative in a large, procedurally crafted world. The final milestone collects all the findings and uses them to implement the artefacts.

In particular, the literature research started by doing a broad search on topics related to the four pillars of this thesis. For every piece of research, I would summarise its contents in a file and then extract key words. These would then be placed in a canvas, where the concepts can be linked and grouped forming diagrams to build a network of knowledge. An example of these diagrams is shown in Figure 4.1 With this diagram structure, it would be easier to keep track of the scoping review.

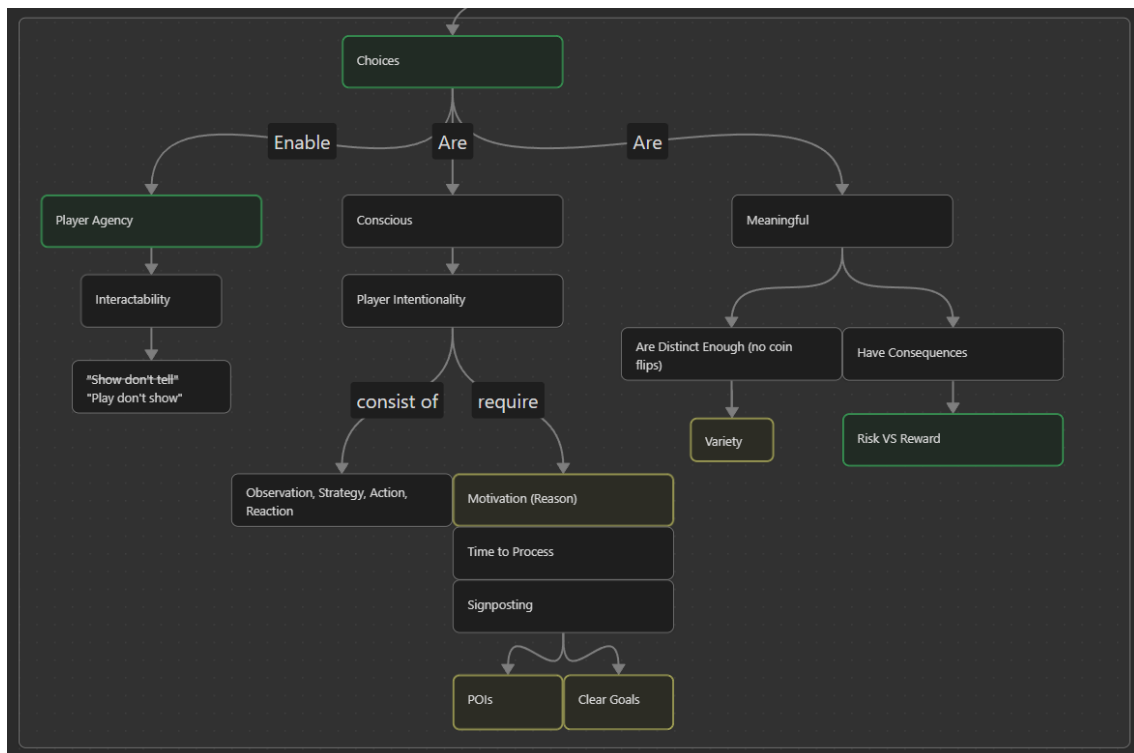


Figure 4.1: Example diagram where concepts related to **Choices** in level design are grouped and linked.

In terms of content, the research process started by finding out what makes a gaming experience and how these relate to open-world games. Then, the research explored publications about tips and insights of level design, which could be linked back to both open worlds and game experiences. A summary of these sources can be found in Appendix C. The research of PCG was much straightforward, exploring the pros and cons of using it against hand-crafted content, or its classification parameters and example methods. The most interesting parts of PCG are the implementation considerations and approach, which can be connected back to level design.

During the research phase, investigation threads were created by looking into the existing diagram and trying to connect two or more concepts. The literature research came to an end after 5 months, when new sources would only provide repeated key concepts that were already present in these aforementioned diagrams, and no more connections could be created. At this step, the gaps in the literature were clearly visible in the diagrams, represented as leaf nodes that showed open questions [Figure 4.2]. This highlighted the premise of this thesis, which explains how there is not much exploration on how level design principles can be applied with procedural generation methods.

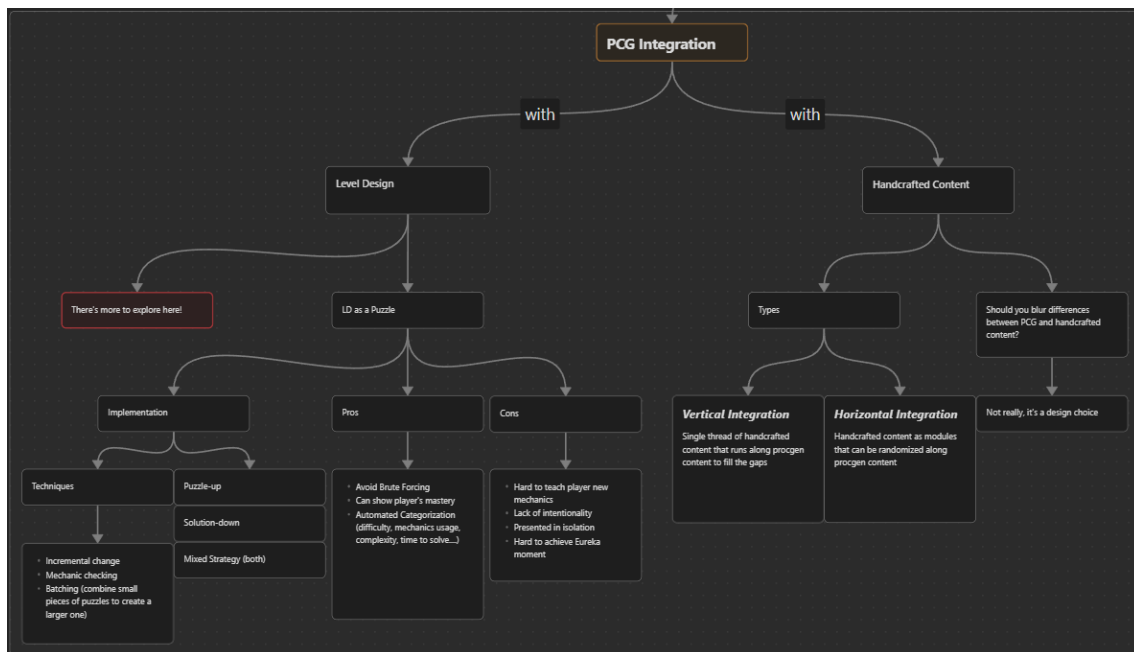


Figure 4.2: Diagram that shows how there is little research background related to the integration of PCG with level design.

4.2 Structure of the Guidelines

This thesis has a strong focus on generic level design research. This is due to the need to define formal rules so that level design principles can be more easily translated into procedural algorithms. Before starting to write down the guidelines, it was important to reflect on what their purpose was. They are supposed to serve as an **actionable** body of knowledge that can be used as a **toolkit**. For this purpose, some additional research was conducted to understand how to create such a practical piece of knowledge. This research led to understanding in what way to categorise all the findings from the literature review and in what way to present them so that they are easier to grasp and retain. Throughout the findings in the literature, several authors presented their own lists of guidelines or patterns [5], [22], [43]–[45]. However, these usually grouped together design practices, considerations, and patterns, mixing tools that are conceptually distinct from one another. For these reasons, the theoretical artefact of this thesis is divided into the following sections:

- **Scope and Context.** This section explains the nature of the guidelines with several points: guidelines are not a manual and they do not focus on a specific game genre. This section also explains what the expected level of expertise is required from the designers to find the guidelines useful.
- **Design Considerations.** This section explains how to put the designer's specific game in context by presenting some game design questions they should answer themselves when starting their projects.
- **Common Design Practices.** This section goes through some common practices professional level designers apply in the industry.
- **List of Guidelines.** This section collects the most relevant concepts and design patterns from level design, and presents them in a particular way to be memorable, practicable, and measurable.
- **How to Apply in Production.** This section reorganises the list of guidelines in such a way that they follow a typical development pipeline.

Each individual guideline was structured to present its definition, some techniques and considerations to apply it, its effects and counter-effects, some metrics and validations, and real industry examples that do or do not implement it. This provides a set of knowledge that is understandable, actionable, and measurable, and that is grounded in real-life examples. Additionally, each guideline includes a section that explain its connection to other guidelines since they all work together like a network of concepts, not as individual and isolated tools. To create the list of guidelines, all the key concepts collected in the first milestone were grouped by proximity into cohesive nested blocks. The title of these blocks would then become each individual guideline.

An important consideration is that the *shape* in which you present your knowledge has different purposes and trade-offs. That is why the list of guidelines is presented in a specific order, and then they are revisited in a different order to understand how to apply them in production. The first sorting is useful as a presentation of the concepts to understand them and progressively and conceptually connect them in the reader's mind. Another useful representation was the one used in the original guidelines diagrams, as seen in Appendix A, but these are harder to translate to a textbook format, so they were used as a stepping stone instead. In the written guidelines it is harder to visualize how all these concepts link to each other, but with the original diagram that relation is not lost. This shows how the representation of knowledge of this artefact is not the only correct and universal way to do it, and other researchers or professionals can differ in this kind of presentation.

One specific example is the video from the industry professional P. Field [5], from which many different concepts were extracted for this thesis. As seen in Figure 4.3, the *order* in which concepts are presented in the video is completely different to the order in which these same concepts are introduced in the guidelines. However, this order does make sense for the video, since all the concepts are linked to a spatial and timely progression, in the same way they belong in the final product of a video game.

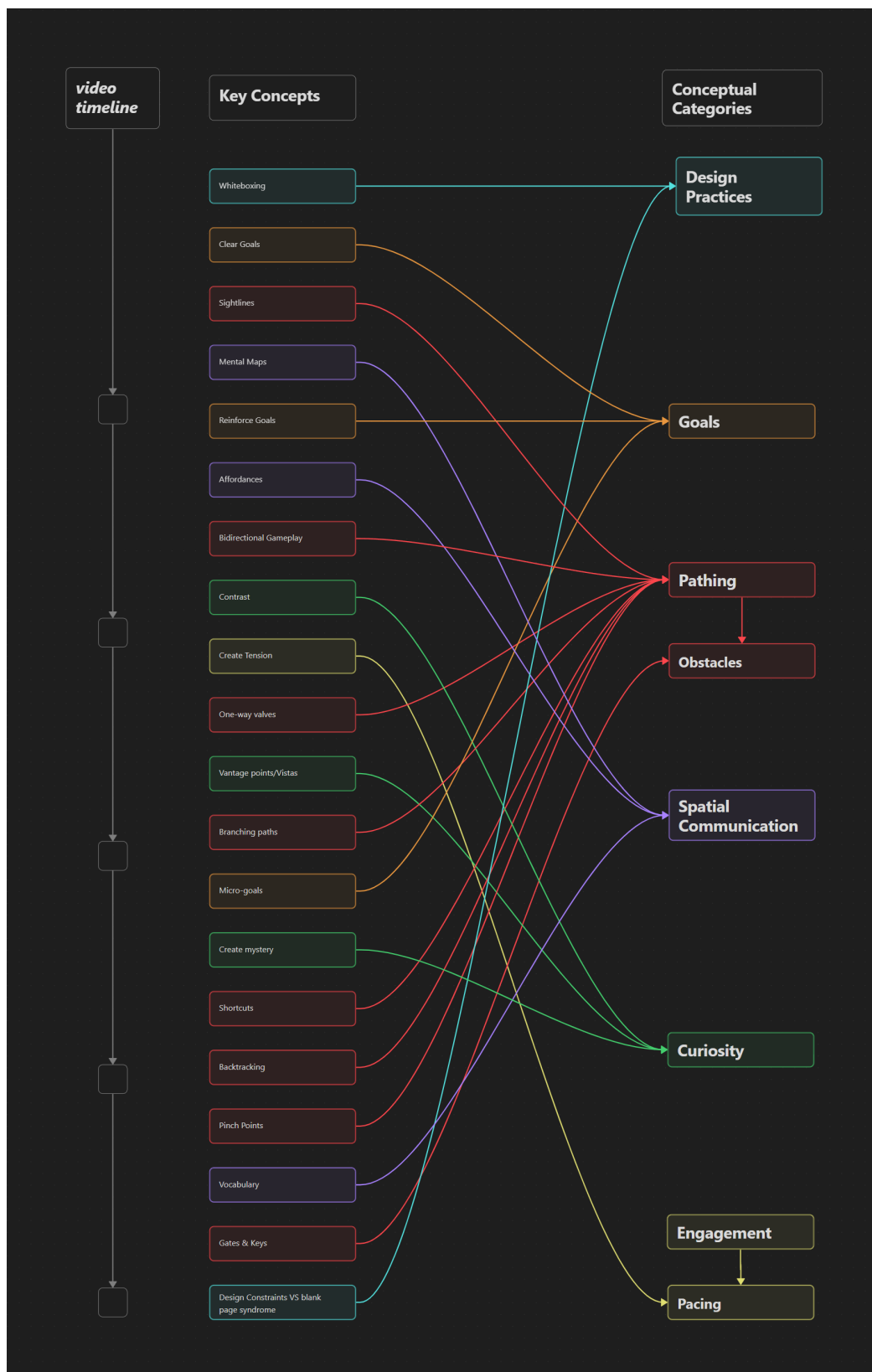


Figure 4.3: Visualization of the chronological order in which level design concepts are presented in Peter Field’s video about spatial communication [5], where each concept is connected to its respective guideline or practice.

4.3 Construction and Purpose of the Prototypes

The design and construction of the prototype was heavily dependent on the results of the guidelines. Explaining these design choices and the purpose of the prototypes rely on being knowledgeable about the key concepts of the guidelines, and for this reason, it is encouraged to read through the **artefact** [1] before continuing with this section.

The main reason to implement the first prototype was to transcribe some of the guidelines into algorithms. Along with this, another reason was to put to practice several of the design considerations and the development pipeline explored during the research phase. The main idea was to **design** the prototype using the guidelines, and to **construct** it using PCG. The design process started by selecting several key concepts, like *Goal Layering*, *Goal Motivations*, *Variety*, **POIs**, **Obstacles**, and more. With this selection, specific game details and algorithms could be defined to be implemented, such as influence maps, biomes, spatial distributions, specific gates and keys, and the implementation of graphs to account for **Pacing**, *Challenge*, and *Sightlines*.

After connecting guidelines concepts to design choices and algorithms, two major blocks were going to shape the prototype: the **probability maps**, and the **progression graph**. The former would use several techniques like recursive noise layering and distribution sampling to achieve design properties like **POIs**, *Challenge*, *Variety*, and *Goal Layering*. The latter would be focused on selecting locations for progression and **Pacing**, and create a game structure to follow the *Pathing* guideline. However, this prototype came with a costly secondary implementation. The mechanics and all the systems required for the minimum gameplay took most of the effort. This, along some technical complications that arose from the 3D nature of the space, led to discarding this prototype and starting the second with a different perspective.

The second prototype focuses strictly on the idea of **translating** design guidelines and concepts to *algorithms*, with no playable outcome at the end. This prototype envisioned to create a map representation of the design choices, but generated procedurally. For this prototype, the approach consisted of focusing on a wide variety of procedural methods and techniques (which could cover several categories in the classification [Section 2.4.3]), and linking them to the appropriate guidelines that could be implemented with them. The selection included specific methods, like noise layering, distribution sampling, CCL, and A*, but it also focused on specific types of input and data representation like sketch-based algorithms, parameters, seeds, and grammars, among others.

5

Results: Theoretical Artefact

The theoretical artefact consists of a structured summary of all the findings from the research on level design. This knowledge is presented in a textbook format, and it can be found in **this webpage** I created [1]. The following sections provide some context on the purpose of the guidelines, a brief overview of their contents, and an exploration on how they are related to the rest of the pillars of this thesis.

5.1 How the Guidelines Were Conceived

First and foremost, it is crucial to point out that the guidelines were designed and structured with the objective to work as an independent piece of knowledge. This means that their content just focuses on level design, and they show no connection to the research of this thesis on the topics of open-world games or procedural content generation. The intent with the guidelines was to create a **self-contained** foundation for level design knowledge. Then, the connection of level design with PCG, and open worlds is explored in this text, without cluttering the theoretical artefact with the goals of this thesis.

It is also important to note how level design shows many different properties and practices depending on the game genre it is applied to. The considerations a level designer has to take into account for stealth games are completely different from shooters, platformers, or exploration games. In that regard, the guidelines try to be **genre agnostic**, and present all of its concepts without assuming the nature of the game the designer wants to create.

Now, focusing on the presentation of the guidelines in the webpage, there are a few points to talk about. The overarching structure of the guidelines was already explained in Section 4.2, but here we can have a look at the specific details of the implementation in the webpage. The first thing to acknowledge is its **interactability**. This web presents a tree hierarchy with all the pages that the user can visit, and where nested sections are *collapsible*. The current page the user is reading is *highlighted*, and page colour coding is used to help **navigation**. Additionally, a **table of contents** is present in the pertinent pages (the home page, each guideline page, and the references). This table of contents is also both interactable and highlights the current section the user is reading.

The guidelines are conceived to serve both as a learning tool, but also as a toolkit. To serve as a **learning tool**, each guideline introduces all the pertinent concepts, explains them in detail, shows how to implement them, and finally it explains how they interact with one another. To be **verifiable**, the guidelines are grounded in real industry examples, and provide metrics so that the designers can check if they are properly implementing them. Finally, to serve as a **toolkit** each guideline presents its *tree diagram* representation with a summary of all the key concepts, so that users can quickly check and refresh what these are without reading through the in-depth explanations when they already are familiar with them.

A final note on the presentation of the guidelines is that, although all the concepts and explanations are backed by solid sources, these are not shown as in-text citations. This is because there is no single source that explains only one concept, and none of the concepts are explained by only one source. These show a many-to-many relationship, and having a list of citations for each concept introduced would only clutter and difficult the reading of the explanations. A ‘References’ chapter is included in the web (same as Appendix C) as a counter measure, where each source shows a summary of its contents and lists what key concepts it contributed to.

5.2 Overview of the Guidelines

This section focuses on the content of the guidelines rather than the presentation. Here is a proposed definition for level design: “Level design describes the practice of shaping **spaces** and distributing game elements to communicate interaction affordances and **guide** the players throughout an intended **game experience**”. The guidelines are based on this definition, and from each highlighted term a category is created. The contents of the guidelines are divided in the following three categories, as seen in Figure 5.1:

- **Game Design.** Although game design belongs to a different role in the development team, level designers are in charge of portraying the game’s intended experience. For this reason, they have to be knowledgeable about these topics. The guidelines under this category explain game design concepts related to **Goals** (objectives) and **Engagement**, but also explore the role of a level designer in these aspects.
- **Guidance.** The guidelines under this category present tools and theory that can be used to influence the players behaviour and desires. These tools are explained in the guidelines of **Points Of Interest**, **Pathing**, and **Curiosity**.
- **Spaces.** The guidelines under this category explain the overall structure of spaces, how to create them, and how to communicate the player this information. These concepts are explained with the guidelines of **Spatial Structures** and **Spatial Communication**. Some concepts that did not make it into the artefact due to scope constraints *suggest* that there might be a third guideline under this category called **Spatial Navigation**, which would explain how players react and interact with spaces, but that is left for future work.

Some of these guidelines, like **Engagement** and **Pathing**, are related to concepts that have a deep background and insights of their own. That is why the explanation of these concepts is delegated to sub-guidelines, so that they can be explored extensively.

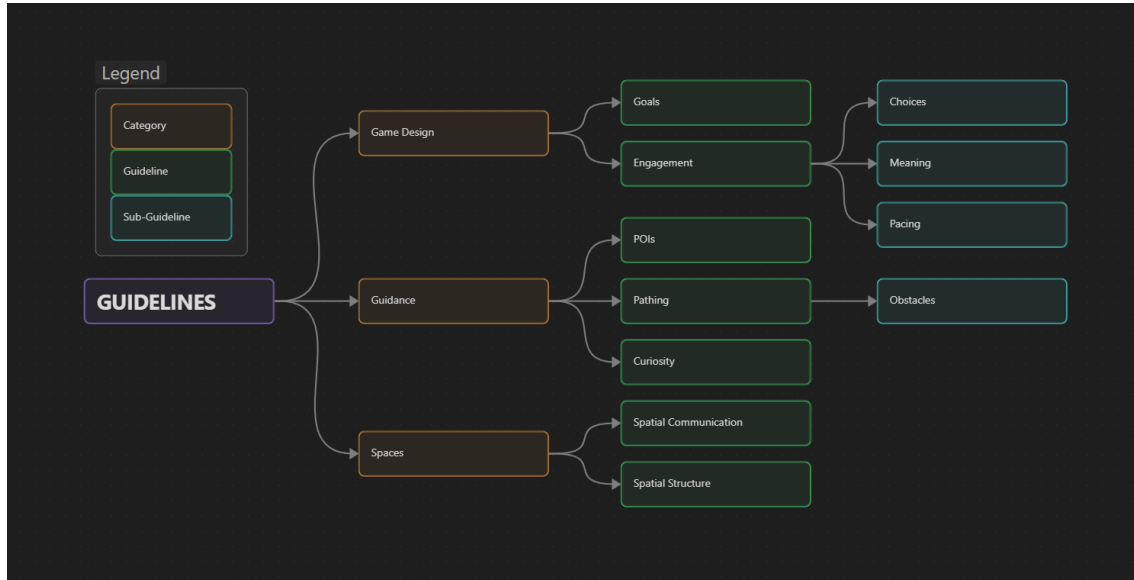


Figure 5.1: Overview of the structure of the guidelines.

As an example, here is a summary of the contents of the first guideline: **Goals**. The overview diagram can be found in Figure 5.2. Firstly, the definition of **Goals** is similar to the one provided in the background about gaming experiences (Section 2.1), explaining the concepts of the *MDA Framework*, *Desires*, and *Goal Negotiation*.

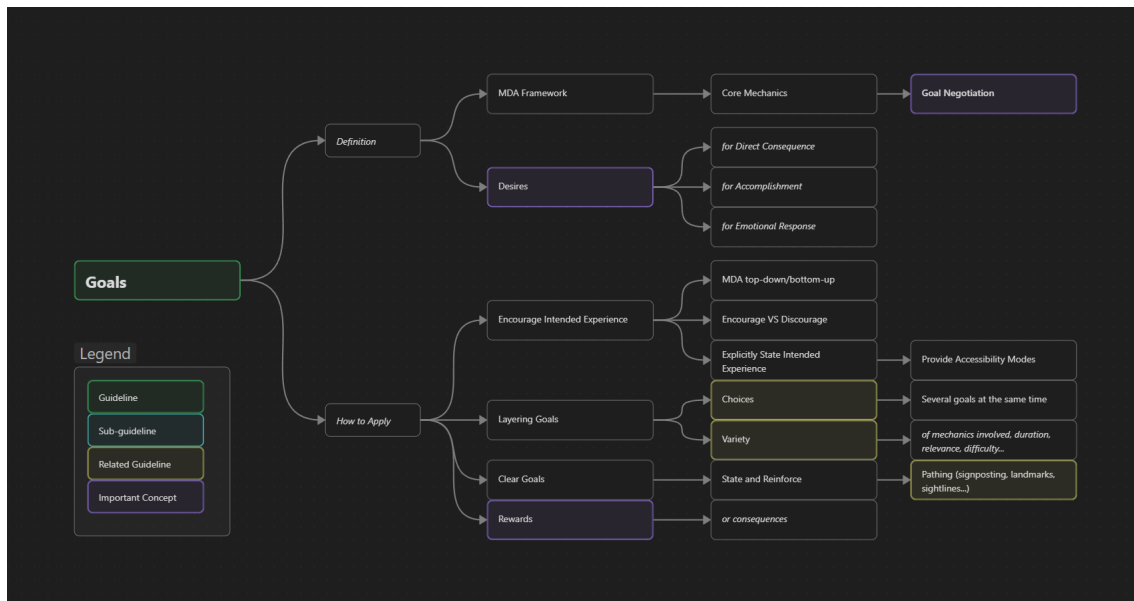


Figure 5.2: Overview of the structure of the **Goals** guideline.

The more interesting bit of the guideline explains how a level designer should implement these game design considerations. The first topic explained is how level designers should encourage the intended experience rather than discouraging or disabling an unwanted one. This topic also explains how to construct these experiences through different approaches with the MDA framework. Then, the guideline explains how goals should be *Clear*, *Layered*, and always offer *Rewards*. These concepts can then be tied to other guidelines like **Choices**, **Engagement**, and **Pathing**. This guideline presents the theory that explains how to use the practical tools provided by other guidelines, like *Variety*, *Signposting*, *Landmarks*, *Branching*, and more.

Some real industry examples that master the implementation of their intended experience are *Doom (2016)* and *Bloodborne (2015)*, where the ‘glory kill’ and the ‘rally’ mechanics respectively *encourage* a more aggressive playstyle by *rewarding* players with health or ammunition. A game that has a very explicit implementation of *Clear Goals* is *The Last of Us (2013)*, where the player’s objective is always linked to some *Landmark* the player has to reach. These *Landmarks* are frequently reinforced throughout *Sightlines* so that players know where they are meant to go.

The validation measure provided in this guideline follows one of the common practices explained in another section of the guidelines, where goals are tracked using a ‘living documentation’. This consists of **listing** all the goals, **organising** them chronologically and spatially, and **checking** that they meet all the criteria, i.e. having *Rewards*, frequent *Signposting*, and enough *Layering* with other goals.

5.3 How Level Design Connects to the Rest of the Pillars

This section explores how the level design findings summarised in the guidelines contribute to the research of this thesis. It explains how level design is connected to the pillars of game experiences, open worlds, and procedural content generation. For that matter, it is advised that you are familiarised with the guidelines [1] and their key concepts before you continue reading.

5.3.1 Level Design in Open Worlds

While the connection of level design with game experiences is easily explained through the definition of **Goals**, there is no explicitly stated connection with open-world games. This connection arises from their definition, as explained in Section 2.2. First of all, some of the guidelines belong to properties that inherent to open worlds. These properties include *Variety*, *Uncertainty*, and *Emergent Gameplay*, all of which belong to the **Engagement** guideline. Open-world games are also described through the nature of their **spaces**, which are non linear. This can be directly tied to the concept of *Player Agency* and *Branching*, which are explained through the **Choices** guideline. As an extension, **POIs** and *Vistas* also strongly contribute the previous guideline. With large spaces comes the need of player orientation, which can be achieved through *Sightlines* and *anchors* or *Landmarks*.

These elements of *Guidance* are essential for open-world games, but some other techniques like *Breadcrumbs* and **Obstacles** also support the intended experience, since they influence the player's *Desires* to follow a more structured progression. These tools also serve to *Reinforce Goals*, which is important to prevent the player from getting lost and disengaged. Another form of orientation for players is through their *Cognitive Maps*. In open worlds, we can help players create these mental maps by following the technique of *Nested Hierarchies* (**Spatial Structures**). Open-world games can also benefit from linear level design techniques (**Pathing**), since these can be applied in the lower levels of the hierarchies to build more traditional levels that make use of *Valves*, *Branching*, *Funnelling*, *Backtracking*, and more.

With these, essentially every corner of level design is covered, having these tools applied in specific ways to support the intended experience. This shows how open world games not only show characteristics that are compatible with level design, but they require most of these tools to successfully align the player's experience with the designer's intended one.

5.3.2 Level Design with Procedural Worlds

Procedural generation also shares with open worlds the concepts of *Variety*, *Uncertainty*, and *Emergent Gameplay* (**Engagement**). These are the most heavily explored characteristics of PCG when applied to video games (e.g. procedural dungeons in rogue-likes), but they work particularly well with open worlds like *Minecraft* (2011). However, the most interesting concepts from a more traditional level design (*Guidance* and *Spaces*) are broadly disregarded by designers. We already explored how these are important to create open-world games that align with a specific intended experience, so now we can explore how to implement them using PCG. What follows is a series of example methods that can be used to implement the guidelines with procedural rules.

The approach to implement procedural open worlds can be based on the level design practice of “building on uneven terrain” [5]. This practice is based on the theory of designing levels with a given set of constraints (spatial in this case) to avoid the *blank canvas syndrome* and achieve more natural-looking results. With PCG, we can first implement a terrain generator with the specific characteristics we need, like hills, valleys, water sources, biomes, etc., and then populate the results with game elements that provide the gameplay opportunities.

To do so, the first step could consist of using ‘Distribution Spawn Maps’. These can be 2D textures that represent the scene, where specific locations are selected at random with a given set of constraints so that each spawnable element (forests, towns, enemies, etc.) follows a different distribution. This is a direct implementation of **POIs**, where a distribution can be sparse/dense, group in clusters, or its elements can be scarce/frequent. This is also tightly related to **Goals**. We can add special constraints to these spawn maps for the selection of locations (following the theory from **Spatial Structures**) to have high/low-risk areas, which can dictate the *Challenge* (**Engagement**) at different locations.

To guide and navigate the player through these locations following a structured progression we can implement a *quest graph generator* based on *grammars*. These quests are assigned to *locations*, which can follow constraints determined by the previous spawn maps. In this way, we can implement **Pacing**, but we can also use specific **Curiosity** practical tools as constraints to influence the player’s possible **Choices**. If quests graphs have spatial constraints for the locations of **Goals**, their structure can show *Nested Hierarchies* (**Spatial Structure**). Furthermore, to create the lower levels in the hierarchy we can generate traditional linear levels with the theory of *Locks & Keys* (**Obstacles**), and the tools presented by the **Pathing** guideline.

The implementation of each individual algorithm mentioned is already complex enough. Chaining several of these generators together can be an immensely challenging task, but it has the potential to provide results that have not been seen before in the video game industry. To recapitulate, the algorithms suggested here are: a **terrain** generator (biomes, natural features, play space, etc.), **distribution** spawn maps to determine the possible spawn locations of different game elements, a **quest graph** generator that uses the previous two algorithms as *constraints* to determine where to spatially place the game’s elements and goals, and finally a **dungeon** generator that serves as an extension to the quest generator to create procedural gameplay instances.

This intricate combination of methods covers almost every guideline. However, **Meaning** and **Spatial Communication** are particularly tricky to implement with PCG. These two heavily rely on the usage of *Environment Art* and some abstract concepts like *Psychogeography* and *World Building*. The presence of **Meaning** is debatable when PCG is involved. The start of a deep discussion in Section 7.1.4 explores how this might be due to a possible reduced sense of ‘authorship’ of the generated content. **Spatial Communication** can be partially achieved through the implementation of its rules within the elements that can be spawned. Additionally, *Affordances* can also be implemented through the consistency of the rules and constraints used in the generators (e.g. spawning an abandoned town will always lead to clues that point where the previous inhabitants hid their treasures).

With this chapter we have explored the knowledge base of level design and how it can be connected to both open worlds and procedural content generation. These connections explain how to apply level design principles to open-world games, and then provide an example of how these could be implemented procedurally, in a way that it can align with the designer’s intended experience.

6

Results: Technical Artefact

The objective of the technical artefact was to display a practical example of how one could implement some of the elicited guidelines of the theoretical artefact into a game making use of different procedural algorithms. The first approach consisted of implementing a simple open-world adventure prototype, where modularity would be the key to achieve a substantial amount of content with minimum systems. However, most of the development time went into implementing many systems that were necessary for the prototype to be **playable**, but that had nothing to do with either the guidelines or procedural generation. For this and other technical reasons the first prototype was discarded, and a change of approach was carried out with the second prototype. The second prototype was designed to skip all the gameplay interactions and just serve as a display of a direct translation from guideline to algorithm.

Despite this change of approach, we can get useful insights from both projects, so in the following sections I will go through the main design choices and the design of the algorithms within each prototype.

6.1 Open-world adventure: Floating Islands

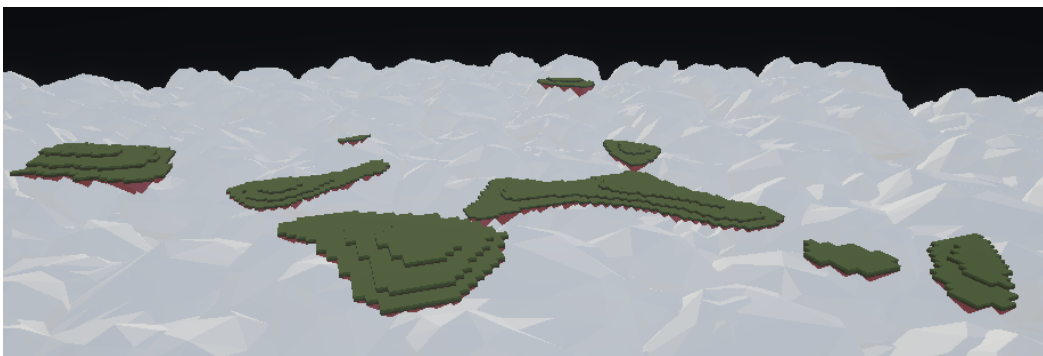


Figure 6.1: An example of a generated cluster of islands.

The development process of this prototype was split into two parts. The first one consisted of the game design and the implementation of many of the systems required to get the game loop running. The second part was in charge of implementing the world generation.

For this prototype, I wanted to follow some of the gathered design practices elicited in the guidelines to recreate the whole process of level design. The design process of this prototype followed J. Burgess’s approach [23] of having a living documentation, where we represent our game details in the shape of a map, a list, and a directory. With this, I could keep track of what were the different locations, where they could be found, and what characteristics defined them. I first started defining what kind of goals the player would have, and what kind of mechanics would be required for that purpose. The game idea consisted of a character that could glide between different floating islands in a finite world enclosed by a dome in the skies and a sea of clouds. All the goals were centred around the concept of exploration, with different narrative motifs behind them. These narrative motifs belong to three different categories:

- **Spatial.** Player could wonder about the boundaries of the world, and explore to break those limits, or understand why they exist.
- **Existential.** Learn about the origin of the world, or find out if they are alone in the world.
- **Practical.** The starting town is running out of resources and they need to expand and search for technological advancements.

With these conceptual ideas the core mechanics came right after, including: the movement (jumping, gliding, sprinting, stamina consumption, gliding leaps, etc), the items (interaction, trading, keys and locks, resource nodes), and the character upgrades (increase stamina or number of leaps). Then, the **game loop** was defined with these mechanics in mind, resolving in the diagram shown in Figure 6.2, where the four core game interactions are highlighted in green. In this diagram you can see how everything connects, and how a typical game experience could be.

The game progression was then defined following the guideline of **Obstacles**, where different items and narrative beats could be understood as *Keys & Locks*. Each of the keys could be found in different biomes, which had different traversal and tools requirements, encouraging the player to engage with the game loop.

After that I worked on the *Master List*, a list of all the different biomes and their properties: the available resources they have, the tools you could discover in them, the visual aspect, the islands layouts (verticality, scale, density, world position, etc), the **POIs**, and a brief summary of the culture of its civilizations.

The relevant part of all these design choices lies in the relation they have with the guidelines. Although the relation is mostly implicit, the following list explains some of these guidelines being integrated in the design of the prototype:

- **Choices.** Players would have to decide what type of activity to engage with, like where to go next, what quest to engage with, what resource to seek, or when to return to the hub to update their progress.
- **Variety.** This is explicitly implemented through the biomes and all their differences of layouts and looks. Challenge would also vary on how difficult the traversal becomes when islands start being smaller and more sparse.

- **Spatial Structure.** This would be defined through a graph implementation, which both affects progression and pacing. It also represents *Boundaries* as unreachable distances. The different traversal mechanics defined the scale of the world, and not the other way around.
- **Points of Interest.** These elements are also explicitly implemented as locations where the player can interact with the game, from resource nodes to towns where they can trade and craft tools.
- **Curiosity.** This is achieved through *Affordances*, which are easily implemented thanks to the modular nature of the generation process. Players would have an easier time reading what elements of the scenery are interactable, and then make better informed choices according to those perceptions.

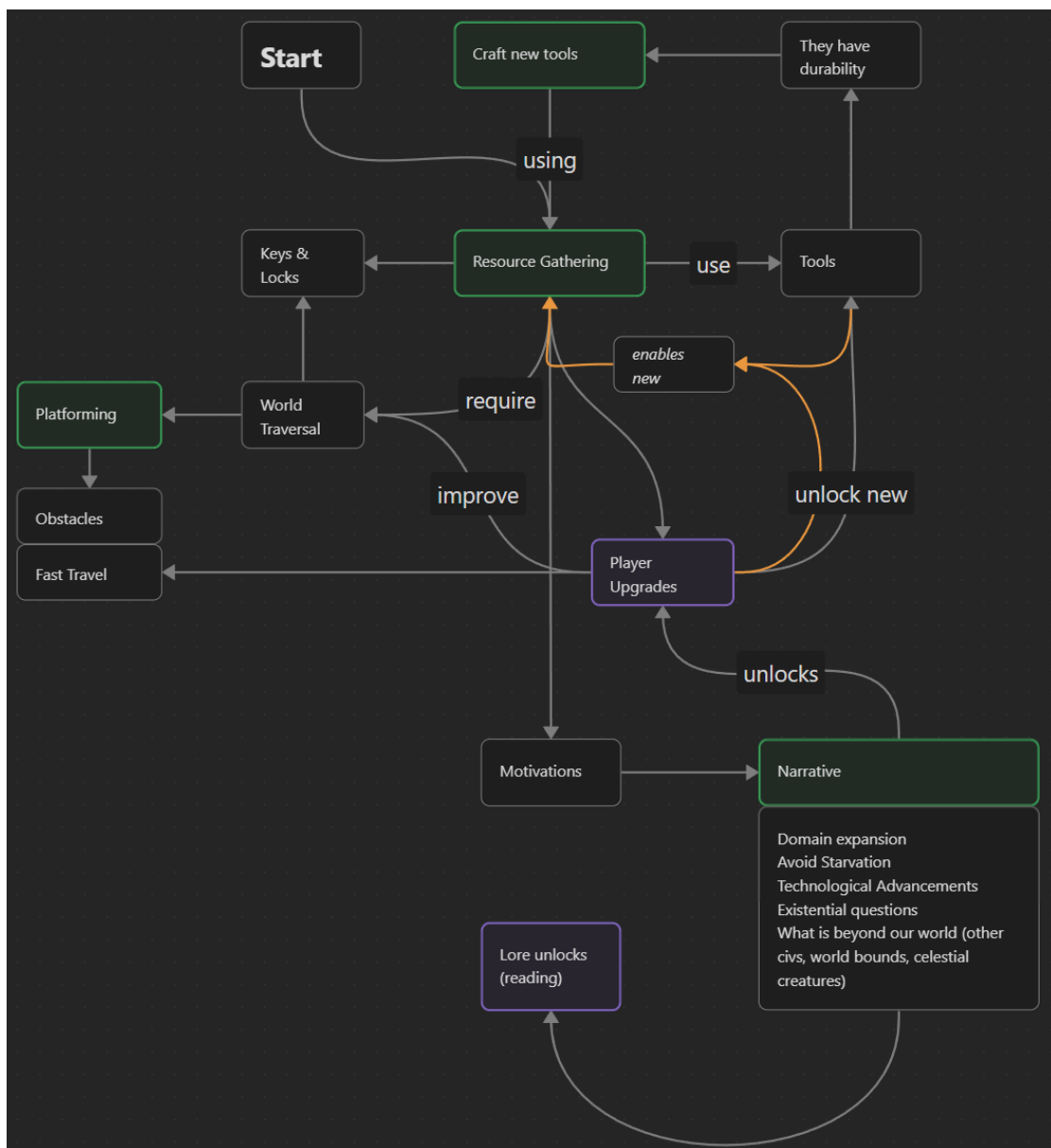


Figure 6.2: Game loop diagram of the first prototype.

When the design phase was finished I translated everything into systems I would need to implement. The movement mechanics were straightforward, but implementing systems like health and stamina, inventory systems, interaction systems, item drops, dialogue boxes, and more turned out to be too much effort and time spent in systems that would not really contribute to the *purpose* of this thesis. At this point I jumped into the world generation, but shortly after running into a technical complication with the calculation of *traversability* I realised how complicated finishing this prototype would turn. At this point I pivoted into the second prototype. However, the world generation of the first prototype still provides useful insights about the complexity and time that can go into implementing intricate rules following a procedural paradigm instead of just hand-crafting a finite world.

The world generation process I designed can be split into two differentiable parts: the generation of **biomes** and the generation of **island clusters**. Biomes are generated in a 3D grid space, where each cell defines what type of biome it is. For each of these cells, a cluster of islands is generated according to how the biome properties dictates.

The biome generation process decides what biome each cell belongs to. If a cell has no biome designated, it will not generate any islands inside. The process is portrayed in Figure 6.3, where a series of samples per biome are spawned in the grid, and then they are expanded to take over adjacent cells and fill the space. Finally, the cells that fall outside the boundaries of the world (outside of the black dome) are discarded. The biome generation algorithm defines a list of properties *per biome*:

- The number of initial samples to spawn.
- The sample growth radius, which defines how many units the sample will grow.
- The world height at which they can spawn (measured in a range).
- The priority they have (which biome survives when one grows inside another).
- The distance from the horizontal centre coordinates at which they can spawn. This can be a maximum radius from the centre (green biome), or a minimum one (which results in a doughnut shape, like the red biome).
- The generation settings for a cluster of islands.

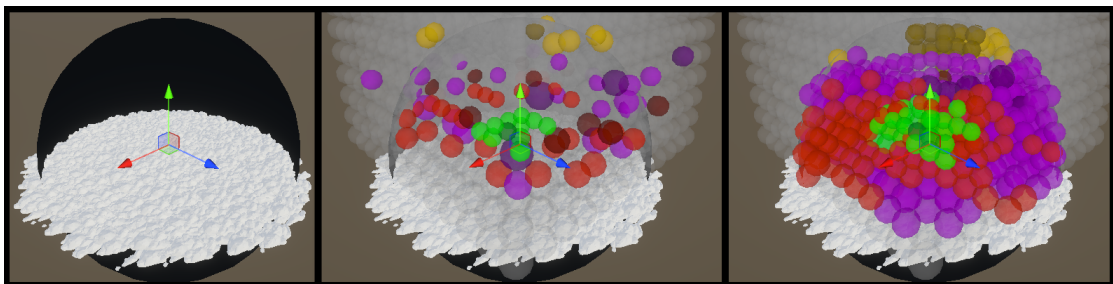


Figure 6.3: Process of biome cells generation inside the world dome of the first prototype. Middle picture represents random biome samples. Right-most image shows these samples expanded to fill the space of the dome.

Once all the *biome cells* are generated, I spawn a cluster of islands for each generated cell. Each biome dictates in which way its clusters of islands are generated according to some properties. A few of these properties are defined as **distributions**. This means that instead of setting a specific value, I define a mean value and a standard deviation (and min/max), so that every time that I need to sample the value I can get a slightly different result (e.g. each island within a cluster can show a slightly different height based on the settings of this property). The generation properties of a **cluster** of islands include:

- **Height multiplier** (distribution), which defines how many world units of difference there can be between the lowest point of the island and the highest.
- **Verticality factor** (distribution), which defines how much elevation difference there is between islands of the same cluster, as seen in Figure 6.4.
- **Perlin Noise settings**. These include the typical X and Y coordinates, the scale factor, and the resolution, along with min/max thresholds. Additionally, a parametrised sigmoid function has been defined to smooth out the generated results, with settings such as min/max factor, exponent (affects steepness of the slope), and distance (defines at which value lies the midpoint of the slope).

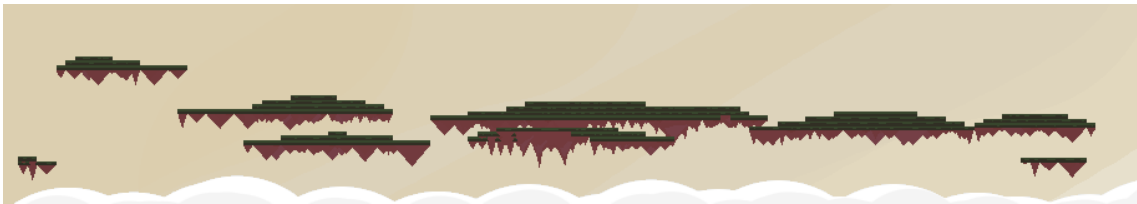


Figure 6.4: Side view of the altitude (verticality) of the different islands in a cluster.

The island generation algorithm is quite more complex than the biome cells. Firstly, a *square* noise texture is generated where each pixel represents the elevation of an island tile (black pixels indicate blank tiles). Then, the pixels outside the circle circumscribed in the square are discarded to reduce the effect of islands looking cut off at the edges.

With this texture, a Connected-Component Labelling (CCL) algorithm is run to detect individual islands and operate them separately. Then, within each island, tiles of **equal height** are merged into squares. The same process is repeated for the remaining tiles, but this time with rectangles. This grouping allows for three benefits: *performance*, since less island pieces have to be instanced; *appearance*, since larger tiles can be appreciated from a bottom perspective; and *practical*, since we now have measurements of the size of the tiles, which allows me to instance large structures on top of the tiles while knowing they will not be clipping adjacent tiles. This process is visible in Figure 6.5. After this, I can apply the verticality factor to randomly alter the altitude of each individual island within the cluster. These steps provide a series of local cluster coordinates, which I can then transform into global coordinates using the coordinates of the biome cell they belong to. With all this information I finally instantiate all the island pieces and scale them to the pertinent calculated tile size.

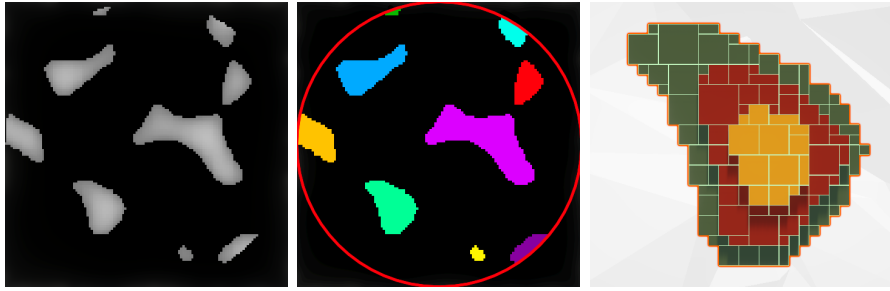


Figure 6.5: (Left to Right) Perlin Noise sample tuned with Sigmoid function; same texture as previous but cutout inside circle and classified with CCL; and tiling of a specific island, where different tile heights are represented by different colours.

From here onwards, the progress stagnated due to gameplay requirements in the generation process. At this point I was required to **measure** distances between islands to generate a **graph** with which I could arrange *progression* and *pacing*. However, this posed a complex challenge with several factors. First of all, the shape of the islands is organic and can generate with a *not-convex* structure. This implies that, depending on the algorithm, it could highly over or under estimate the distance between two islands. Furthermore, to create the reachability graph a single distance value would not suffice since, depending on the **unlocked abilities** of the player, they could traverse the space in different ways and, depending on how much stamina or glide leaps they unlocked, the resulting graph would look different. This means that the graph would have to be **dynamic** and react to player upgrades. Additionally, to understand if a measurement is valid for the player we have to account for height, duration of glide, number of available leaps, and so on. These factors greatly hindered measuring the reachability between islands. Finally, the design of a new mechanic was required to *regain altitude*, since the player can always glide down to lower islands, but it is unlikely they can come back the same way. After all these technical complications the first prototype was discarded.

6.2 World Map: Guidelines to Generation Rules

The purpose of this prototype was to have a one-to-one representation of how some guidelines can be translated into different properties of an algorithm. The approach consisted of creating a **maquette** representation of a slice of playable space, without it actually being playable. This prototype was just in charge of creating the play space, without any of the game mechanics or interactions being implemented. A specific intention with this prototype was to explore several types of PCG methods and techniques. For this reason, I implemented methods with different types of data representation, designer controllability, and input required.

Before getting through the guidelines that are portrayed in this prototype we will go through the **generation pipeline** to put them in context. This prototype uses several textures and arrays in its generation that serve as representations of the calculations before finally constructing the world and instancing all the elements in place. The generation pipeline goes as follows:

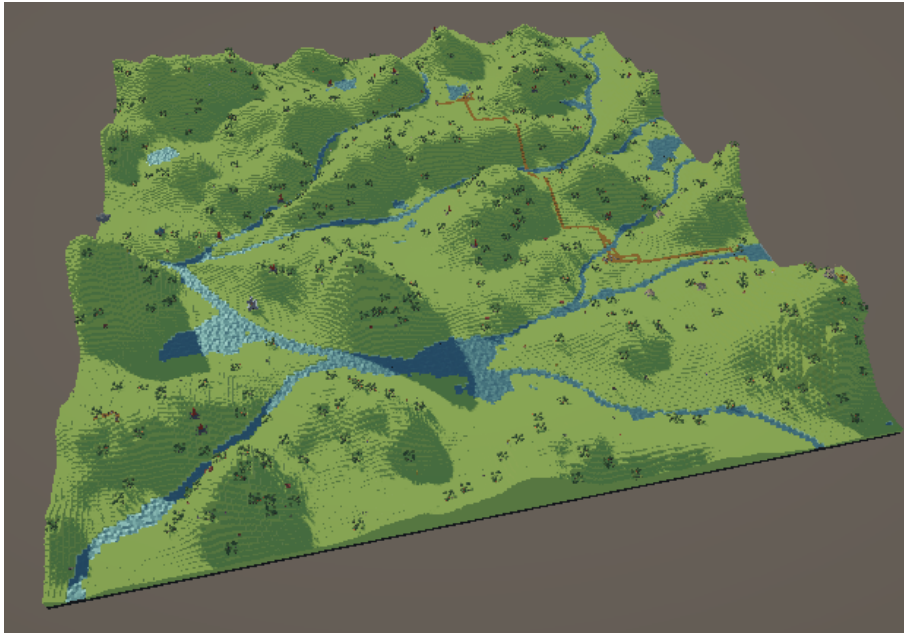


Figure 6.6: Overview of an example generated map.

- **Initialization.** First, the algorithm resets by destroying the previous world if there was one and then it initializes all the required textures and arrays. At this step the algorithm can set the seed of the algorithm if one is provided.
- **Terrain Elevation Map.** The first step of the generation is to create the world elevation (hills and valleys). To do so, 3 octaves of Perlin Noise are layered to achieve a basic effect of terrain.
- **Water Bodies.** The elevation texture can generate small pockets of water, but for the generation of rivers (or any kind of water body), the level designer can input a hand-painted texture, as seen in Figure 6.7. The ‘water bodies’ texture is generated by mixing this input texture with the water locations generated in the elevation map. With this new texture, the algorithm calculates the ‘near water bodies’ texture (with a parametrised radius). The ‘water bodies’ texture is then used to mask the elevation texture, and then the ‘near water’ texture is used to smooth the terrain down to water level ($Y=0$).

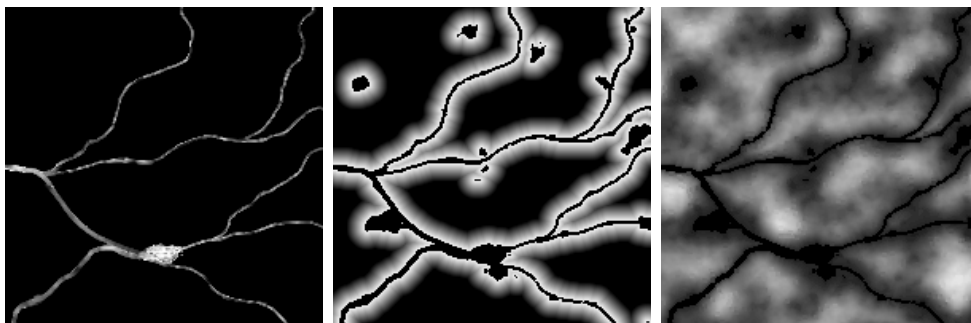


Figure 6.7: Hand-painted water bodies texture (Left), ‘near water’ texture (Center), and ‘terrain elevation map’ (Right) after applying the ‘water bodies’ mask.

- **Inclination Map.** Once the ‘terrain elevation map’ is ready, an additional texture is generated by calculating its steepness (using the gradient). This ‘inclination map’ is later used to decide the spawn probabilities for a variety of different types of elements. You can find this texture in Figure 6.9.
- **Towns Spawn Map.** At this step the probability spawn map of town buildings is generated. This is where special generation rules linked to game design start to apply. This spawn map uses the information of the ‘near water’ texture and the ‘inclination map’ to define the spawn probabilities of towns. These rules dictate how towns are more likely to spawn near bodies of water and in rather flatter areas.
- **Enemies Preparation.** For the generation of enemy locations two additional textures are required: difficulty and intensity. These are generated through a single layer of Perlin Noise, where the level designer has parametrized control. The **difficulty** texture defines how challenging the type of enemy encounter is, by using different tiers of enemies, with increasing health and attack values. This texture also uses elevation as a factor, so harder challenges can be found at the top of the hills. The **intensity** texture will determine what type of challenge the player will have to face, from single wandering enemies, to enemy camps, to boss fights. This is visualised in Figure 6.8.



Figure 6.8: Intensity Map (Left); Enemies Spawned (Middle); and Enemy Ranking (Right). In the intensity map, a brighter value represents an area with more intense enemy challenges. In the world map, enemies are highlighted in red.

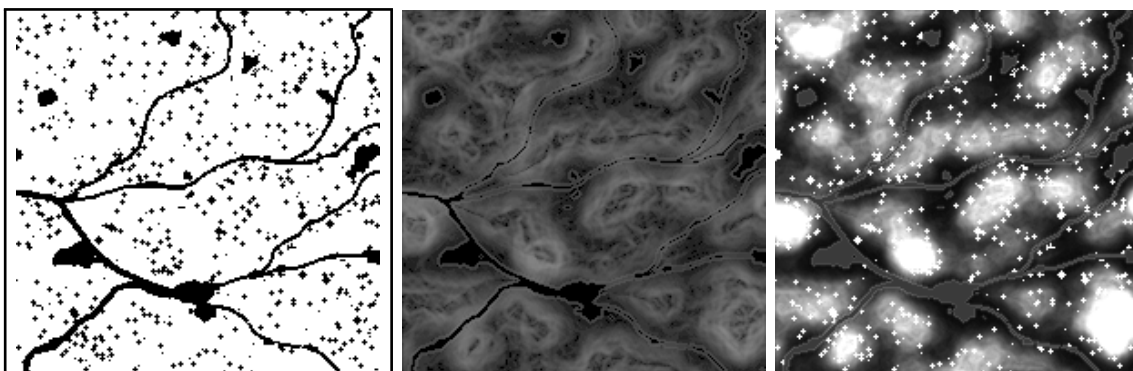


Figure 6.9: (Left to Right) Global map (instancing availability); Inclination map (derivative of elevation); and A* Weights for creating paths between towns.

- Generate Locations.** Different types of **POIs** have different approaches when their location is generated. They all avoid invalid spawn positions by using the ‘global map’ (Figure 6.9) as a mask. When a POI’s location is decided, it is marked as no longer available (black) in this map. Some of the POIs are selected at random from a pool of prefabs, while others (*unique locations* like player spawn, main goal, and enemy dungeon) are used as a list to spawn each only once. The structures that occupy more than one tile of space can **alter** the ‘elevation map’ so that they will not clip through the ground. The most interesting locations generated are the towns. These use the ‘towns spawn map’ to select locations where **clusters** of buildings of different sizes can be spawned. This way, building locations are not entirely random, but instead they clump together. To sample from the probability maps (Elevation, Inclination, etc.) I make use of a method that transforms the pixels of the texture into a **Cumulative Distribution Function (CDF)**, where brighter pixels have a proportionally higher chance of being selected. Once a value has been sampled, it gets removed from the CDF and the total sum value gets updated. The sampling of enemies follows an additional rule that uses both the ‘intensity’ and ‘difficulty’ maps to decide what type of element is going to be spawned, since they have different footprints and they might impact the ‘elevation map’.
- Town Paths.** Once the town locations are generated, some dirt paths are created to connect them. Individual building are sampled and paired up with another one that is inside a minimum and maximum distance, so that paths are not created between buildings in the same town or towns that are in opposite corners of the map. The pathing algorithm uses A*, which requires a list of weights. These weights are stored in a texture (Figure 6.9), that is generated by using the elevation, inclination, global maps, and water bodies. Paths will avoid cliffs and water when possible, and they will never run into an obstacle. This behaviour can be seen in Figure 6.10.

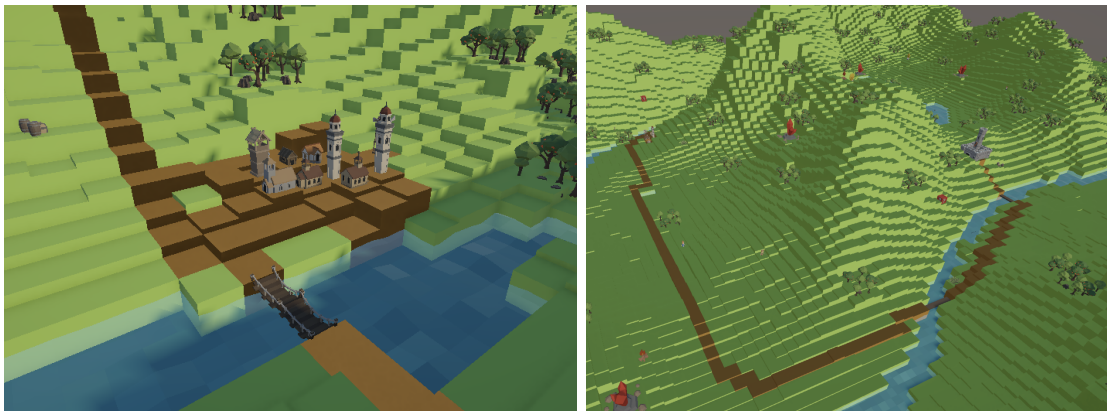


Figure 6.10: Display of how towns tend to spawn in low inclination areas near the water, and paths are generated avoiding water and hills.

- **Instantiation.** All the previous steps worked with different data structures like textures and arrays of coordinates. These are now used in the final step to instantiate all the elements in the world. The elevation, water bodies, and towns paths textures are first used to generate the land. With these, each tile can select to be one out of the 3 possible types: land, path/bridge, or water. Then, the algorithm runs through all the saved locations for POIs and instantiates them with a random orientation. You can visualize these results in Figure 6.11.

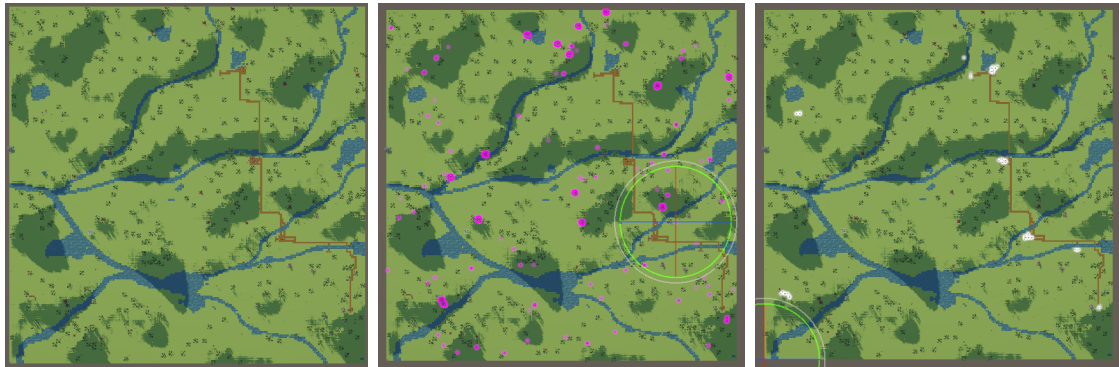


Figure 6.11: (Left to Right) Top-down view of the world map, then highlighted POIs in magenta, then highlighted towns in white.

The structure of these generation rules not only creates a playground where the player can spend some time at, but they can also provide a specific gameplay experience with designer intent behind. Tying the spawn location of all these level elements (NPCs, enemy encounters, quest markers) to spatial generation rules, like altitude or proximity to water, leads to specific gameplay opportunities that are supported by the level design guidelines. Furthermore, by defining goals that encourage the exploration of the environment, the player has an excuse to visualize their surroundings and come up with their own *desires*.

These goals can be formally defined through specific verbs such as “go to, find/explore, defeat, interact, or acquire”. These verbs can then be attributed specific subjects or nouns to start constructing a **grammar**. You could go to or find a location, you could find a level element or an enemy, you could acquire a resource or a new quest, etc. These structures, along with some properties and constraints that could integrate pacing, difficulty scaling, or spatial rules, can become an effective grammar to further dictate the world generation in the shape of a *graph*. However, due to time constraints this quest generation system was only developed theoretically. This means that guidelines like **Pathing**, **Choices**, and **Spatial Structure** (graph generation) were not implemented into this prototype, but have a strong potential. The guidelines that do have representation in this second prototype include:

- **POIs.** The usage of probability spawn maps and CDFs naturally work with POIs due to their parametric nature. Defining properties like density/sparsity, clustering, and uniqueness, and tying these to their level of interest and footprint or visual impact makes for a compelling implementation of the guideline.

- **Pacing and Challenge.** The prototype follows the simple approach of the ‘difficulty’ and ‘intensity’ maps to define these two properties. It would be interesting to see a more complex system where these maps are used along the progression graph with the *quest generation system*, but that is to be explored in the future work.
- **Variety.** The modularity of these systems enables the addition of new elements to a potential game without much effort by just defining the type of spawn requirements they have and setting up their spawn distribution parameters. One example could be adding new types of enemies, where instead of just goblins you could have lizards, knights, or ghosts, with their respective variants for different levels of intensity and difficulty.
- **Pathing: Vistas.** This one particular pathing technique is achieved organically by tying the spawn constraints of different level elements to the elevation of the map. Some POIs are designed to spawn on top of the cliffs, while towns will mostly spawn near water in the valleys. This facilitates a gameplay loop where players can be commanded from a town to go complete a task at one of these POIs, where they can then scout their surroundings and find new objectives.

7

Conclusion

This thesis explored how to implement level design in procedural open-world games while aligning with an intended game experience. The research process of these four pillars followed an exploratory research by applying a scoping review. The pillars of gaming experiences, open worlds, and PCG had their own individual research phase since they are more self-contained, while the exploratory research of level design served to connect all the previous concepts together. Level design could be linked to game experiences through the concept of *Goal Negotiation*. It could also be linked to open worlds through the specific level design techniques that are used for guidance in open spaces.

However, the scoping review showed how there is a gap in the literature about the integration of level design and PCG, where there is no common ground on what a correct approach would look like. This thesis served to point out this gap in the literature, to explain why this connection is interesting, and to prove its feasibility. This feasibility was explored by first formally defining several level design concepts, and then translating some of them into procedural algorithms in a prototype.

7.1 Discussion

This section explores the takeaways and limitations of the resulting artefacts of this thesis, along with some self-reflection on the work done both for the theoretical and practical artefacts. This section also questions the nature of this thesis and challenges the idea of creating interesting open-world experiences with PCG.

7.1.1 Takeaways of the Guidelines

The guidelines provided in this thesis aim to create an actionable body of knowledge in a textbook-like structure that can serve to learn and be used as a guide to apply level design concepts in a video game. This is based in the theory of *Pattern Languages* [38], where a common vocabulary can serve (designers and researchers in this case) to ease communication and have a representation of common problem-solution pairs. These guidelines are also heavily inspired in the *Sapir-Whorf hypothesis* [37], which explains how ‘*the language that we speak shapes the way we think*’. This can be translated into how designers envision the spaces they create based on the design concepts they are familiar with.

The guidelines also remark the importance of the realisation that different presentations of the same knowledge have different purposes, benefits, and drawbacks. This is portrayed by an example from the literature, from which several concepts were extracted (P. Field [5], Figure 4.3), but where the *sorting* of these concepts is quite distinct from their arrangement in the guidelines. This concept is later reinforced in the guidelines in the section called ‘How to Apply in Production’. This alternative sorting groups the guidelines in such a way that they are easier to apply in a development pipeline instead of just teaching the concepts to the reader.

Another consideration about the guidelines is that they try to encompass a broad field of research, where many of these individual concepts present extensive studies of their own. A problem arises with some parts of this knowledge being **tacit**, which professional developers try to explain in their insights, but which are better understood through practice. Furthermore, the intent of the guidelines in this thesis was not to serve as a final contribution of the research, but as a stepping stone to then be applied through procedural generation. For these reasons, the guidelines cannot be considered a complete set of level design knowledge, since they were not conceived for that purpose, and aiming for that would pose a monumental task. To exemplify, here are some concepts that did not make it into the guidelines:

- Attraction versus repulsion of the player in a space. What are typical elements that can serve for these purposes, and what are the benefits and consequences of using one over the other.
- An exploration of the usage of *Game Design Documents* (GDDs), and their alternatives as different types of live or in-game documentation.
- Insights on how players usually read the spaces they are presented with. How they can ignore verticality if they are not prompted, or how they can read the space from left to right, and the first thing they see can be the one that sticks with them. An explanation on how some players will try to explore secondary paths before the apparent main path.
- How to effectively implement paper prototyping, as in, designing levels with pen and paper. This refers to the usage of layers, colour values, and line girth to clearly represent the ideas behind the design of a level.

One particularity of the level design research is that all the findings are not directly referenced in the main text, but they are instead explained in a ‘References’ section (as seen in Appendix C). This is due to how many of the sources overlap in the topics they cover and how each individual source often addresses multiple concepts. Having multiple in-text citations for every concept explained would clutter and confuse the reader rather than provide useful information.

Finally, one major takeaway from the level design research is lack of an established foundation. As Jesús Machina observes, level design is a discipline so young that it lacks its own standardized nomenclature, often "borrowing" concepts and terminology from already established fields like architecture [46]. This absence of formal standardization underscores the necessity of developing structured resources, such as the guidelines proposed in this thesis.

7.1.2 Takeaways of the Prototypes

The practical artefacts show a substantial list of limitations. These were considered during the design and implementation phases, but ended up discarded due to scope constraints. The main limitation was that the **guidelines** are *incomplete*, and so the depth of evaluation of the prototypes that can be done with these is quite shallow. Only the first two guidelines have a section explaining their metrics and validation tools, and most of those require either a playable environment or the integration of other underdeveloped guidelines. While user evaluation was considered in the roadmap of this thesis, since a copious amount of time and effort was spent in the first prototype, the **scope** of the second prototype was *reduced*. This implied that no playtesting or surveys could be carried out to test the efficacy of the integration of level design principles with procedural content generation.

However, one specific thing that the prototypes displayed is the **complexity** behind implementing procedural generation algorithms with *abstract* design rules. The prototypes have examples of both implemented and conceptualised algorithms, and the effort and difficulty that they pose. The first prototype displays the complexity involved in the generation of the islands, their variability, their biomes, and their structures. The generation process was complex enough, but at the end it showed how the true complexity arises from the integration of gameplay restrictions, in this case by trying to measure the distance between organic-shaped islands to arrange game progression. The second prototype also displayed the complexity of the implementation of several techniques with the different instantiation rules of the POIs.

The second prototype additionally proposed the implementation of another algorithm. It suggested a first approach to a grammar generation for quests that could follow both the **Pacing** and **Pathing** guidelines. However, implementing such an algorithm could become a thesis on its own. While this was the original end goal of this thesis, the research phase revealed that a more foundational exploration of level design was required before starting with such complex algorithms. Now that we have more solid knowledge of what constitutes level design, we can explore this topic in future work.

An interesting insight from the prototypes is that, given the scope restrictions, the implementation focused on procedurally generating a single plot of land instead of an infinite world. This presents the idea that procedural generation can be used not only to generate infinite worlds but also to automatise the creation of self-contained adventures. This means that PCG can be used as a **designer tool**, and that was exactly the process I followed. I created the second prototype by designing tools with many parameters and different types of designer control. By fine-tuning these, I achieved a result that I deemed interesting. This usage can be extrapolated to a real context, where maybe not the whole process has to be automatised, but maybe these tools can be used to create samples where level designers can later take over and hand-craft a specific experience.

As a final remark, while the prototypes did not prove a successful translation from guideline to algorithmic rules, they showcase the potential of such implementation in a future and more developed technical artefact.

7.1.3 Validation

The exploratory nature of this thesis enabled the creation of a foundational taxonomy of level design, while establishing the background for its connection with game experiences and its integration with procedural generation. However, this approach also resulted in an expansion of scope that constrained the use of formal evaluation methods. These include the use of metrics or user studies to validate the results, which were ultimately delegated to future work. Despite this, the application of research-through-design enables a process of self-evaluation, which can be applied both to the research process and to the artefacts of this work.

As explained in Section 4.1, the research process followed a roadmap divided into three milestones: literature review, field research and design, and implementation and evaluation. A fourth milestone could also be considered if the writing of this thesis is included, which took approximately three months. However, while the second and third milestones took 1.5 and 2 months respectively, most of the time was spent on the first milestone, which spanned five months. This literature review phase focused primarily on level design, which explains the strong presence of the theoretical artefact.

While research on game experiences, open-world games, and procedural content generation could be explored through a limited number of well-chosen sources, level design revealed the absence of a unified and formalised body of knowledge. This is apparent in Appendix C, where each source presents only a subset of the concepts collected in this thesis. Furthermore, some of these concepts are grouped differently in the literature, whereas the guidelines distinguish between design considerations, practices, guidelines, and operational resources.

The creation of the level design concept tree presented in Appendix A underwent an iterative process. During the review of sources, concepts were initially scattered and were gradually grouped and connected into cohesive categories. This process of connected thinking later allowed the identification of gaps in the literature when attempting to link level design to the other pillars. Finally, transforming these diagrams into guidelines not only provided a useful way of presenting knowledge in an applicable format, but also revealed how different forms of representation serve distinct purposes. While the guidelines are structured in a textbook-like and digestible format, I personally found the diagram representation more useful as a cognitive map of the concepts.

While the guidelines lack formal validation, they are grounded in reliable sources, including peer-reviewed papers, theses, books, and professional insights such as GDC talks. Although their structure has not been evaluated, they have personally helped me develop a deeper understanding of the concepts and operationalise them for practical application rather than leaving them as abstract ideas. Finally, regarding the practical artefacts, most of the implemented guidelines follow explicit translations from theory into system design. However, the creation of vistas has a more subjective nature. A collection of the generated vistas can be found in Appendix B.

One interesting realisation that comes from the research of this thesis is that the sources exploring how to integrate level design with PCG often do so while focusing on the concepts of *Unpredictability* and *Variety*, and some times *Pacing*. And that is if they are not explaining how to implement procedural algorithms with a ‘puzzle design’ approach, or explaining how to integrate hand-crafted content. These are sensible explorations since these concepts are inherent properties of PCG. However, although these are very relevant concepts related to **Engagement**, an exploration of the guidelines from the *Guidance* category could provide experiences that show a game structure with designer intentionality. This is why it would be interesting to use quests generators to implement **Pathing** techniques.

7.1.4 Meaning in Procedurally-Generated Worlds

Meaning is a crucial sub-guideline that serves **Engagement**. It refers to the sense of purpose that players experience while playing, and the feeling that what they are doing in the game matters. Meaning can be born from the **connections** players experience, and these can be cognitive, personal, emotional, or spatial. Level design particularly focuses on the spatial category, which occurs when meaning is embedded into spaces [47]. The particular tools level design uses to create this type of meaning are *Environmental Storytelling* and *Psychogeography*.

This becomes an interesting discussion topic when introducing procedural content generation in the development process, since these two aforementioned tools heavily rely on narrative and author intentionality [48], which are two particular weaknesses of PCG. This complicates the embedding of meaning in spaces when using this kind of algorithmic approaches.

In terms of narrative, non-authored generated content may reduce perceived value or engagement in players. This is further to be explored, but a running debate topic with the recent rise of generative Artificial Intelligence (AI) discusses how the perception of authorship can affect how meaning is constructed. Experimental evidence indicates that consumers evaluate AI-generated art more negatively than human-made art, suggesting that the source of creative output impacts perceived meaning [49]. In terms of author intentionality, it becomes a diffuse topic in the context of PCG. Here, there is still authorship over the intended experiences, but the control is indirect. Game developers design the generation rules instead of the specific instances, which can lead to unexpected or unintended results that deviate from the author’s intent.

This far, we have only referred to meaning as something that must be provided by the author, but interpretative meaning changes the focus from the author to the consumer. This reframes PCG from a model that lacks author intent to one that redistributes where the origin of meaning lies. This framing could work better along the use of PCG since, despite the lack of precise control, the author can instead create the possibility space that can lead to opportunities for the players to invest the worlds with their own meaning. Author intentionality can be important here too since, although meaning here emerges from the consumer, their interpretative engagement can be reduced if they perceive this lack of human authorship [49].

From a more pragmatic perspective, the implications of this discussion can help level designers have a better understanding of the implications of meaning, and how to ultimately and effectively integrate it into their worlds and systems. This discussion helps explain the trade-off between authored control and emergent meaning, although a more in-depth exploration of the topic would be required for the fully developed guideline.

7.2 Future Work

This thesis widely explored the literature of a broad range of topics, trying to find an answer to a question that requires answers to many more fundamental questions. In doing so, it presented two artefacts. However, due to the huge scope this thesis was aiming for, these ended up with many loose ends which I will explain now.

Most of the research that belongs to the future work lies on the theoretical artefact: the guidelines. First of all, they would require to be fully developed, in the same way that it has been done for their first two entries (**Goals** and **Engagement**). A particularly interesting addition would be an in-depth catalogue of examples that show how different implementations of each guideline evoke different game aesthetics. One example could explain how the *frequency* of **Goals** creates different sensations, e.g. in a game where you are a bartender, a lack of customers could expose the player to apathy, while an infinite hoard of customers could induce stress. To improve the guidelines, it would also be useful to have clear examples of their presence in games, which could be achieved through crowdsourcing, since one has to be familiar with a game to state if it implements said guidelines. Furthermore, the missing concepts and design practices mentioned in the takeaways (Section 7.1.1) should be integrated in the final product. The guidelines should also be validated through different means, like an MDA analysis over the listed games, or surveys to explore the effect these patterns have in the players themselves. Professional developers should also be contacted to directly provide their insights and opinions of the structure of this knowledge. Additionally, different displays of this knowledge could be explored by rearranging their conceptual categories and their sorting. One particular concept that shows how this might be necessary is *Uncertainty*. This concept is used in two guidelines: it is a tool used to apply guidance through the player's **Curiosity**, but it is also an attribute of **Engagement**. This concept showing up in two different guidelines can be used as a hint to indicate that a different arrangement of the concepts could help in their understanding.

As for the technical artefacts, their future work involves revisiting their current limitations listed in Section 7.1.2. This could involve either implementing a playable prototype with metrics and validation systems like playtests, or the implementation of a quest generator with graphs that account for spatial **Pathing** techniques, such as *Vistas*, *Shortcuts*, *Valves*, *Sightlines*, *Branching*, and more.

7.3 Closing Remarks

Coming back to the research question: ‘How can we implement level design in procedural open-world games following the intended game experience?’. The answer that can be provided with the findings of this thesis is quite fragmented. For this purpose we need to acknowledge the four pillars and understand the connections between them. We first explored how game experiences could be defined and analysed through the MDA framework. Then, we described what constitutes an open-world game by defining common characteristics they share, like exploration and non-linear objectives. Here we identified that ‘open world’ does not constitute a game genre but rather acts as a qualifier on top of other genres. Level design acts as the linking pillar that unifies everything together. It connects to game experience through the concept of *Goal Negotiation*, where the act of distributing the game objectives throughout the world incentivises this targeted experience. The level design guidelines can then be used to implement the intended experience by following the pertinent rules that apply to open-world games. Finally, the implementation of these rules would be applied by translating them into algorithms thanks to the classification parameters and list of methods provided for PCG.

This could work as a tentative answer, since the guidelines have been constructed around the definition of game experiences, and all of them describe how they impact player experiences. Particularly, the guidelines provided a concise but flexible definition on how to construct **Goals** and **Engagement**, while at the same time they offered example consequences of implementing them one way or another in the ‘How to Apply’ and ‘Counter Effects’ sections. Additionally, this thesis briefly showed some examples on how these rules can be translated into algorithms by using procedural generation. With this we can see how there is an answer to the question, but it will always depend on the type of experience the game designer wants to construct. There is also no definitive way to translate the level design rules into algorithms. Their implementation heavily depends on what rules you are combining, and what is your intended game experience. A complete set of examples for each of the guidelines could help refine this answer.

While a conclusive answer to this research question has not been provided, this thesis remarks the importance of this topic and points out the existing gap in the literature. Furthermore, this research question opens up many more related questions that are worth exploring, such as: ‘How can we effectively teach level design?’, ‘What level design tools are more compatible with PCG?’, or ‘What are the implications of PCG in the meaning of worlds?’. Additionally, the work of this thesis sets the foundation of knowledge from which to build further investigations in many different directions. This could consist of deepening the current research of level design and extending the guidelines with player surveys, industry professionals’ insights, and examples with different game experiences in mind, or taking a more technical approach and providing implementation examples making use of PCG.

Acronyms

AI	Artificial Intelligence		Aesthetics
BOTW	(Zelda) Breath of the Wild	NPC	Non-Playable Character
BSP	Binary Space Partitioning	POI	Point Of Interest
CCL	Connected-Component Labelling	PCG	Procedural Content Generation
CDF	Cumulative Distribution Function	RNG	Random Number Generator
GDC	Game Developer Conference	RPG	Role-Play Game
MDA	Mechanics, Dynamics,	RtD	Research through Design
		WFC	Wave Function Collapse

Ludography

- | | |
|-------------------------------------|-------------------------------------|
| – .kkrieger (2004) | – Goat Simulator (2014) |
| – Adventure (1979) | – Hytale (EA 2026) |
| – Assassin’s Creed saga (2007) | – The Last of Us (2013) |
| – Bloodborne (2015) | – The Legend of Zelda series (1986) |
| – Colossal Cave Adventure (1976) | – Minecraft (2011) |
| – Comanche: Maximum Overkill (1992) | – Noita (2019) |
| – Death Stranding (2019) | – Pitfall! (1982) |
| – Doom (2016) | – Satisfactory (2024) |
| – Elden Ring (2022) | – Spelunky (2008) |
| – The Evil Within series (2014) | – Subnautica (2018) |
| – Firewatch (2016) | – Teardown (2022) |

Bibliography

- [1] R. Martín, *Level design guidelines*, Github Pages, <https://zephyr-62.github.io/Level-Design-Guidelines/>, 2026.
- [2] N. G. J. Hughes, “Understanding specific gaming experiences: The case of open world games,” <https://etheses.whiterose.ac.uk/33608/>, Ph.D. dissertation, University of York, 2023.
- [3] R. Hunicke, M. Leblanc, and R. Zubek, “A formal approach to game design and game research,” in *Proceedings of the Challenges in Game AI Workshop, Nineteenth National Conference on Artificial Intelligence*, 2004.
- [4] N. E. M. Vickery and P. Wyeth, “Exploration in open-world videogames: Environment, items, locations, quests, and combat in the witcher 3,” in *Proceedings of the 34th Australian Conference on Human-Computer Interaction*, New York, NY, USA: Association for Computing Machinery, 2023, pp. 310–318, ISBN: 9798400700248. [Online]. Available: <https://doi.org/10.1145/3572921.3572926>.
- [5] P. Field, “Spatial communication in level design,” in *Develop Digital*, <https://www.youtube.com/watch?v=AKeUZVikPV8>, 2020.
- [6] R. I. Zubek, *Elements of Game Design*. Cambridge, MA: MIT Press, 2020.
- [7] J. Burgess, *Skyrim’s modular level design*, Game Developers Conference, 2013. [Online]. Available: <http://blog.joelburgess.com/2013/04/skyrim-modular-level-design-gdc-2013.html>.
- [8] C. W. Totten, *An Architectural Approach to Level Design*, 2nd ed. CRC Press, 2019.
- [9] E. Adams, *Fundamentals of Game Design*, 3rd ed. Berkeley, CA: New Riders, 2014.
- [10] T. Fullerton, *Game Design Workshop: A Playcentric Approach to Creating Innovative Games*, 4th ed. Boca Raton, FL: CRC Press, 2018.
- [11] J. Schell, *The Art of Game Design: A Book of Lenses*, 3rd ed. CRC Press, 2019.
- [12] M. e. a. Hendrikx, “Procedural content generation for games: A survey,” *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 9, no. 1, Feb. 2013, ISSN: 1551-6857. DOI: 10.1145/2422956.2422957.
- [13] D. Grey, “When and why to use procedural generation,” in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 1.
- [14] M. R. Johnson, “Integrating procedural and handmade level design,” in *Level design: processes and experiences*, CRC Press, 2017, ch. 11, pp. 217–242.

- [15] M. R. Johnson, "Worlds," in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 10.
- [16] L. O. Valencia-Rosado and O. Starostenko, "Methods for procedural terrain generation: A review," in *Pattern Recognition*, Cham: Springer International Publishing, 2019, pp. 58–67, ISBN: 978-3-030-21077-9. [Online]. Available: https://www.doi.org/10.1007/978-3-030-21077-9_6.
- [17] B. Bucklew, "Algorithms and approaches," in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 26.
- [18] R. Montserrat, "Procedural content generation for video games, a friendly approach," *Level Up [Game Dev Hub]*, 2025, <https://www.levelup-gamedevhub.com/en/news/procedural-content-generation-for-video-games-a-friendly-approach/>.
- [19] G. Smith, "Pcg overview," in *Level Design Processes and Experiences*, CRC Press, 2017, ch. 8.
- [20] S. Björk and J. Holopainen, *Patterns in Game Design*. Hingham, MA: Charles River Media, 2005.
- [21] C. Alexander, *The Timeless Way of Building*. New York, NY: Oxford University Press, 1979.
- [22] M. Barclay. "My level design guidelines." <https://mikebarclay.co.uk/my-level-design-guidelines/>. (2016).
- [23] J. Burgess, "Level design planning for open-world games," in *Level design: processes and experiences*, CRC Press, 2017, ch. 12, pp. 243–272.
- [24] J. Machina. "Define primero, diseña tu nivel después." (2026), [Online]. Available: <https://youtu.be/1nq-FemmLrM>.
- [25] S. Rogers, "Levels," in **Level Up! The guide to great video game design**, John Wiley & Sons Inc, 2014, ch. 9.
- [26] T. X. Short. "Level design in procedural generation." (2014), [Online]. Available: <https://www.gamedeveloper.com/design/level-design-in-procedural-generation>.
- [27] A. Khalifa, F. de Mesentier Silva, and J. Togelius, "Level design patterns in 2d games," in *2019 IEEE Conference on Games (CoG)*, 2019, pp. 1–8. DOI: 10.1109/CIG.2019.8847953.
- [28] J. Grinblat, "Designing for modularity," in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 4.
- [29] L. Welton, "Aesthetics in pcg," in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 3.
- [30] G. Programmer. "Procedural generation in games." (2022), [Online]. Available: <https://generalistprogrammer.com/procedural-generation-games>.
- [31] J. Segree, *The case for procedural generation in puzzle games*, 2024. [Online]. Available: <https://www.youtube.com/watch?v=xqmTp20juZk>.
- [32] B. W. Head and J. Alford, "Wicked problems," *Administration & Society*, pp. 711–739, 2013, <https://doi.org/10.1177/0095399713481601>.
- [33] R. A. Stebbins, *Exploratory Research in the Social Sciences*. Thousand Oaks, CA: SAGE Publications, 2001.
- [34] E. L. Deci and R. M. Ryan, "The "what" and "why" of goal pursuits: Human needs and the self-determination of behavior," *Psychological Inquiry*, 2000.

- [35] L. von Bertalanffy, *General System Theory: Foundations, Development, Applications*. New York, NY: George Braziller, 1968.
- [36] S. Johnson, *Emergence: The Connected Lives of Ants, Brains, Cities, and Software*. New York, NY: Scribner, 2001.
- [37] B. L. Whorf, *Language, Thought, and Reality*. Cambridge, MA: MIT Press, 1956.
- [38] C. Alexander, S. Ishikawa, and M. Silverstein, *A Pattern Language: Towns, Buildings, Construction*. New York, NY: Oxford University Press, 1977.
- [39] D. A. Norman, *The Design of Everyday Things*. New York, NY: Basic Books, 2013.
- [40] J. Zimmerman and J. Forlizzi, “Research through design in hci,” in *Ways of Knowing in HCI*, J. S. Olson and W. A. Kellogg, Eds., Springer New York, 2014, pp. 167–189, ISBN: 978-1-4939-0378-8. DOI: 10.1007/978-1-4939-0378-8_8.
- [41] C. Fosnot, *Constructivism: Theory, Perspectives, and Practice*. Teachers College Press, 2005, ISBN: 9780807745700. [Online]. Available: <https://books.google.se/books?id=rvnFQgAACAAJ>.
- [42] Z. Munn, M. Peters, and C. e. a. Stern, “Systematic review or scoping review. guidance for authors when choosing between a systematic or scoping review approach,” *BMC Med Res Methodol*, 2018, <https://doi.org/10.1186/s12874-018-0611-x>.
- [43] S. Lee, “An approach to holistic level design,” in *GDC*, <https://gdcvault.com/play/1024301/Level-Design-Workshop-An-Approach>, 2017.
- [44] J. O. Fares Kayali, “Level design practices for independent games,” in *Level Design Processes and Experiences*, CRC Press, 2017, ch. 6.
- [45] D. Taylor, “Ten principles for good level design,” in *GDC*, <https://gdcvault.com/play/1017803/Ten-Principles-for-Good-Level>, 2013.
- [46] J. Machina. “Inmersión vs realismo: Cómo diseñar espacios jugables creíbles.” (2026), [Online]. Available: <https://www.youtube.com/live/8Bd1zhFxCus?t=7809s>.
- [47] D. M. R. Johnson, “Meaning,” in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 27, pp. 301–311.
- [48] L. Andreotti and X. Costa, *Theory of the dérive and other situationist writings on the city*. Museu d’Art Contemporani de Barcelona eBooks, 1996.
- [49] F. Tigre Moura, C. Castrucci, and C. Hindley, “Artificial intelligence creates art? an experimental investigation of value and creativity perceptions,” *The Journal of Creative Behavior*, vol. 57, no. 4, pp. 534–549, 2023. DOI: <https://doi.org/10.1002/jocb.600>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jocb.600>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jocb.600>.
- [50] B. Wang, Z. Gao, and M. Shidujaman, “Meaningful place: A phenomenological approach to the design of spatial experience in open-world games,” *Games and Culture*, vol. 19, no. 5, pp. 587–610, 2024. [Online]. Available: <https://doi.org/10.1177/15554120231171290>.
- [51] B. C. João Raza, “The illusion of choice - how to hide linearity of levels,” in *Level Design Processes and Experiences*, CRC Press, 2017, ch. 10.

- [52] D. J. Dormans, “Graphs and cycles,” in *Procedural Generation in Game Design*, CRC Press, 2017, ch. 9.
- [53] E. Games. “Level design fundamentals.” (2023), [Online]. Available: <https://dev.epicgames.com/community/learning/tutorials/3VKJ/unreal-engine-fortnite-level-design-fundamentals>.
- [54] R. Y. Smith. “The level design book.” (2023), [Online]. Available: <https://book.leveldesignbook.com>.
- [55] A. Serr, “Designing radically non-linear single player levels,” in *GDC*, <https://gdcvault.com/play/1026398/Level-Design-Workshop-Designing-Radically>, 2021.
- [56] N. Oueijan, “Stop getting lost: Make cognitive maps, not levels,” in *GDC*, <https://gdcvault.com/play/1027206/Stop-Getting-Lost-Make-Cognitive>, 2022.
- [57] A. Yoder, “Playgrounds and level design,” in *GDC*, <https://gdcvault.com/play/1025884/Level-Design-Workshop-Real-World>, 2019.
- [58] Nintendo EPD, *The making of the legend of zelda: Breath of the wild*, Developer Interview, 2017. [Online]. Available: <https://youtu.be/30jGWna4-Ns>.
- [59] R.-Y. Storm, *Spatial documentation: Gyms, zoos, and museums*, YouTube video, <https://youtu.be/5PJRCz0t7yY>, 2021.
- [60] K. Beachum, “Sparking curiosity-driven exploration through narrative in ‘outer wilds’,” in *GDC*, <https://gdcvault.com/play/1027368/Independent-Games-Summit-Sparking-Curiosity>, 2021.
- [61] A. Beachum. “Designing for curiosity in outer wilds.” <https://youtu.be/vGnce1Dp9BU>. (2023).

A

Obsidian Diagrams

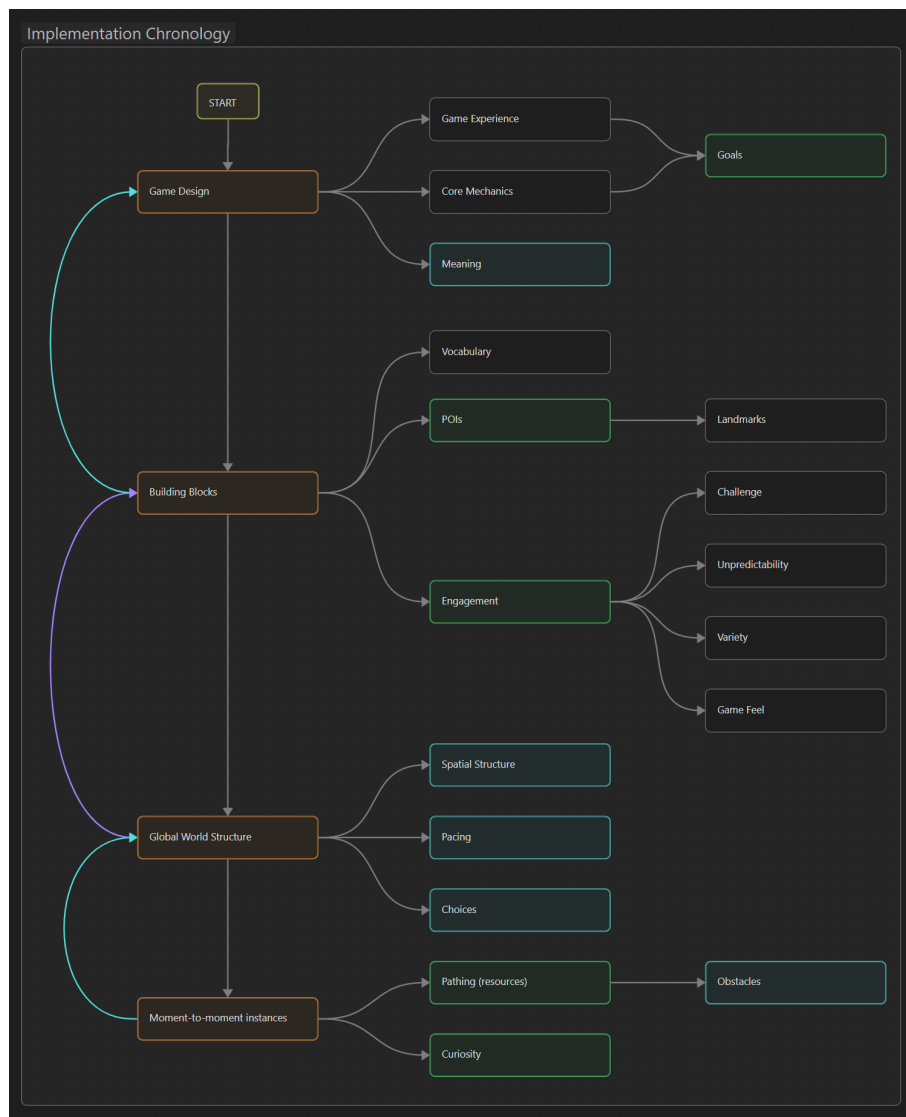
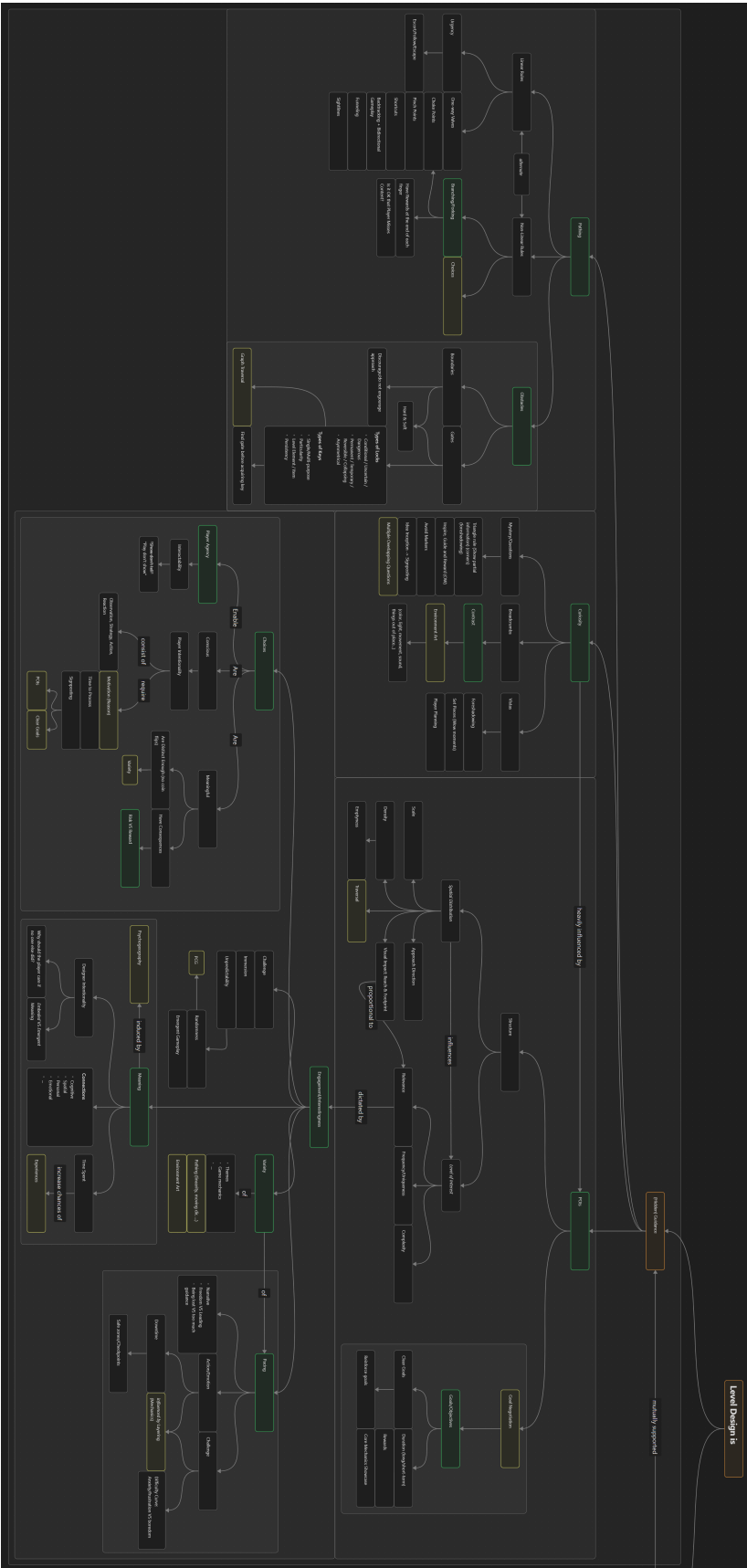


Figure A.1: Diagram that displays in what order all the guidelines can be applied for a specific iterative development pipeline.



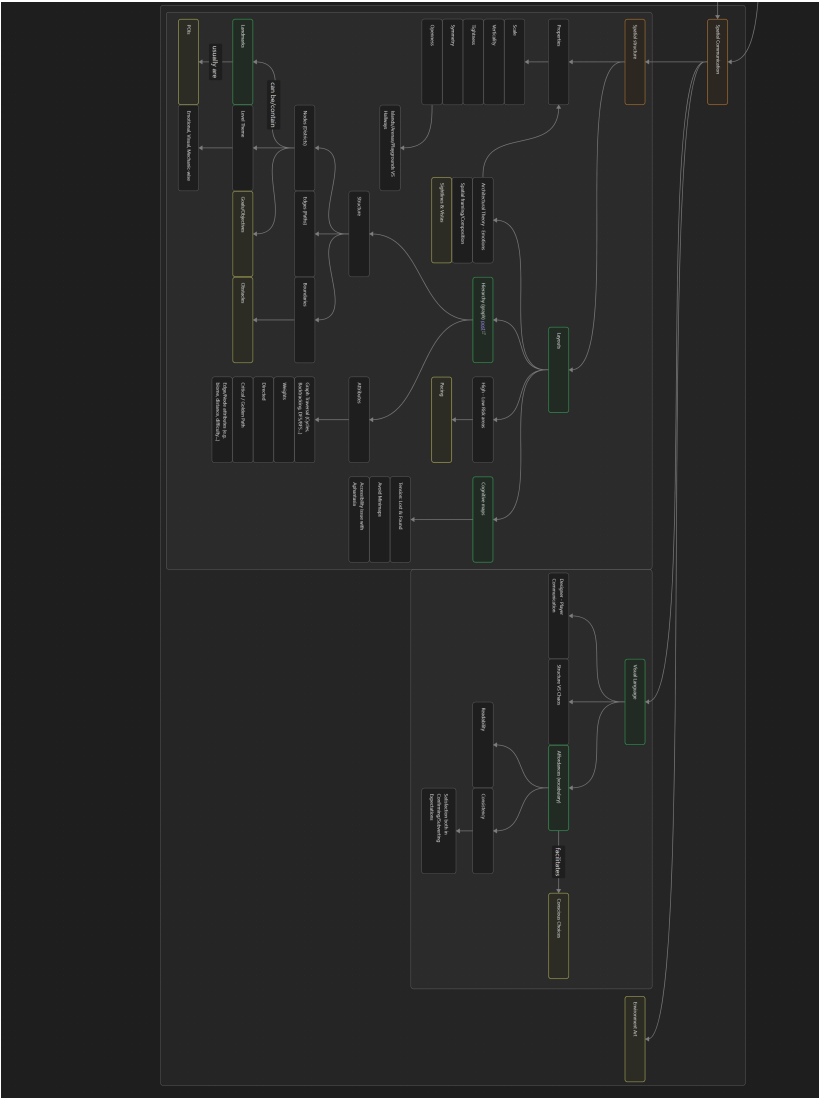


Figure A.3: Level Design canvas [Part II] that displays all the concepts related to spaces.

A.1 Level Design Guidelines

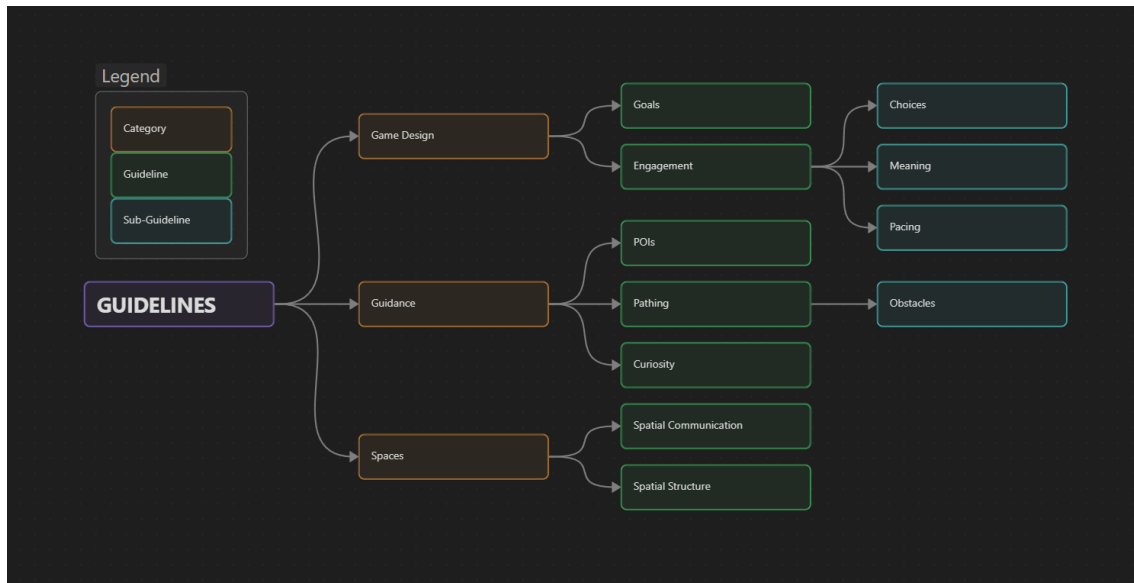


Figure A.4: Diagram that displays an overview of the structure of the guidelines.

A.1.1 Guidelines: Game Design

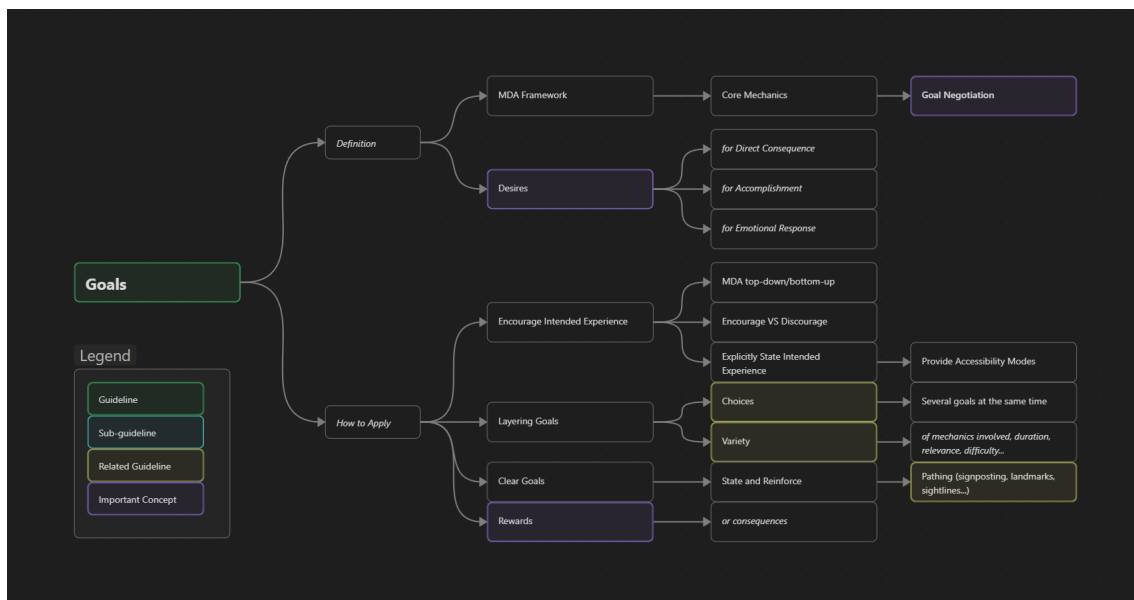


Figure A.5: Diagram that displays an overview of the structure of the **Goals** guideline.

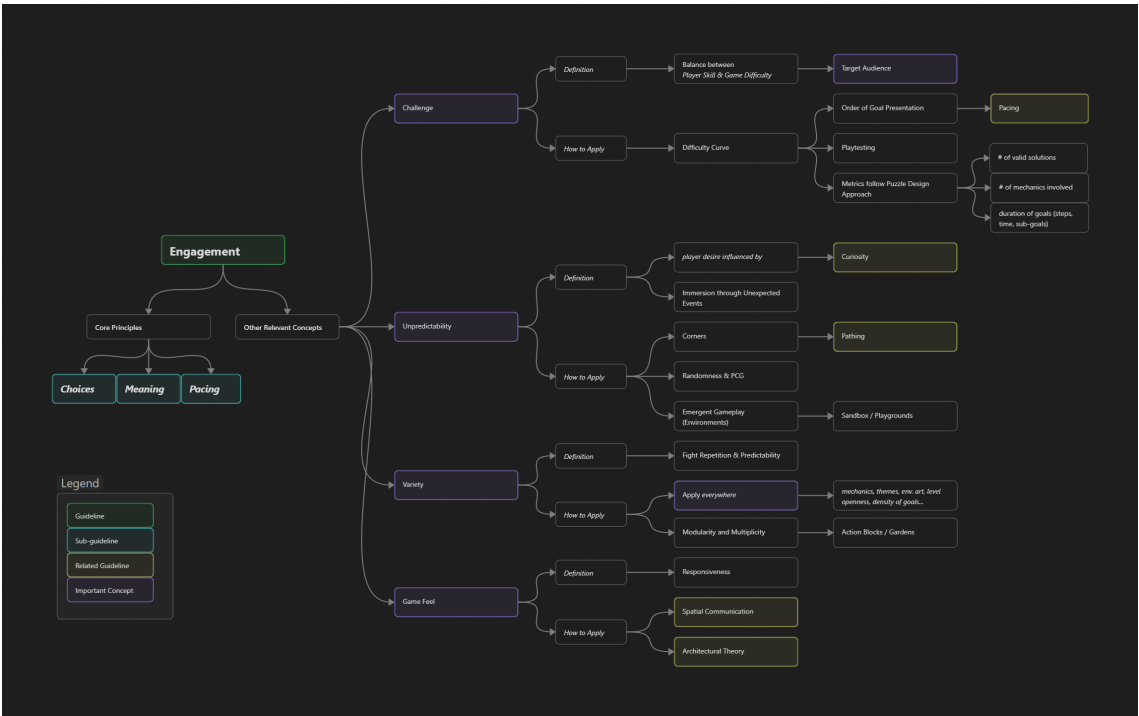


Figure A.6: Diagram that displays an overview of the structure of the **Engagement** guideline.

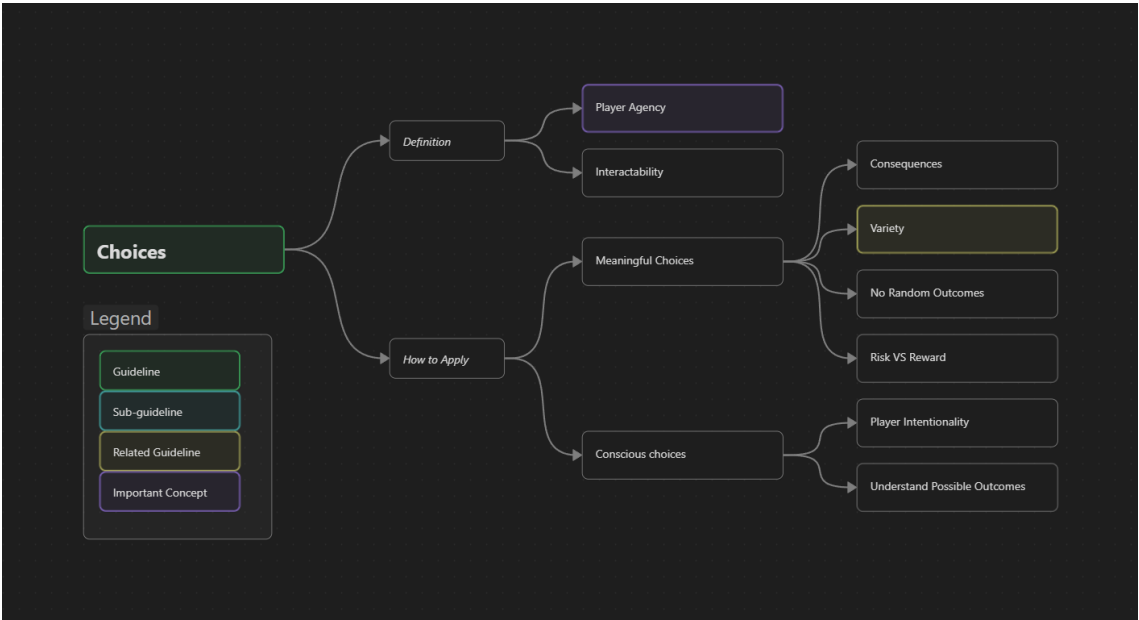


Figure A.7: Diagram that displays an overview of the structure of the **Choices** guideline.

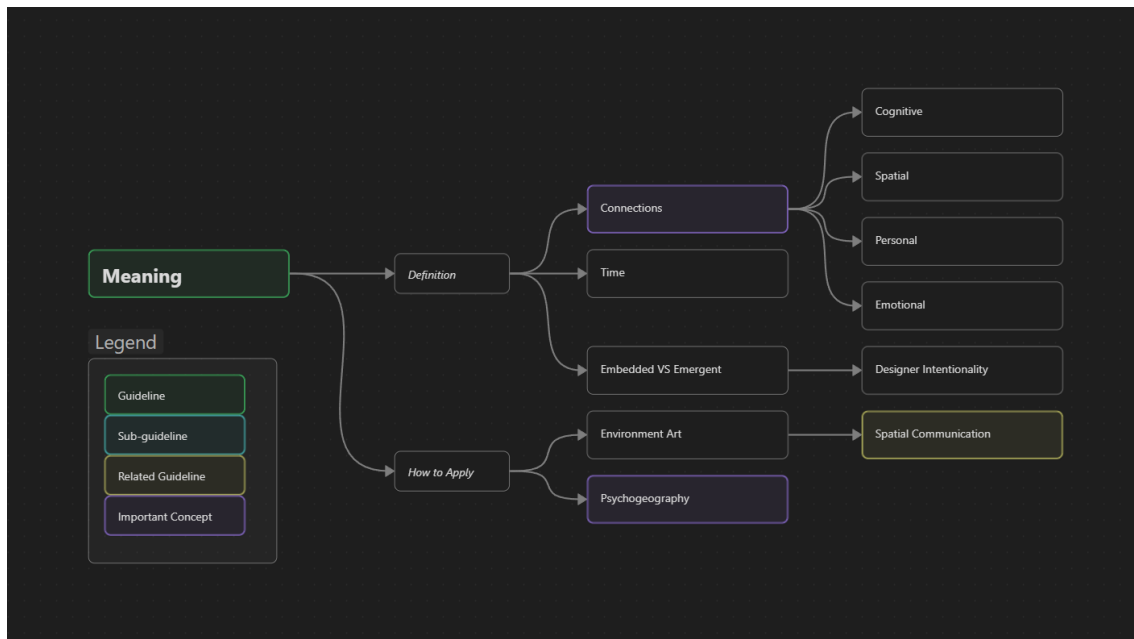


Figure A.8: Diagram that displays an overview of the structure of the **Meaning** guideline.

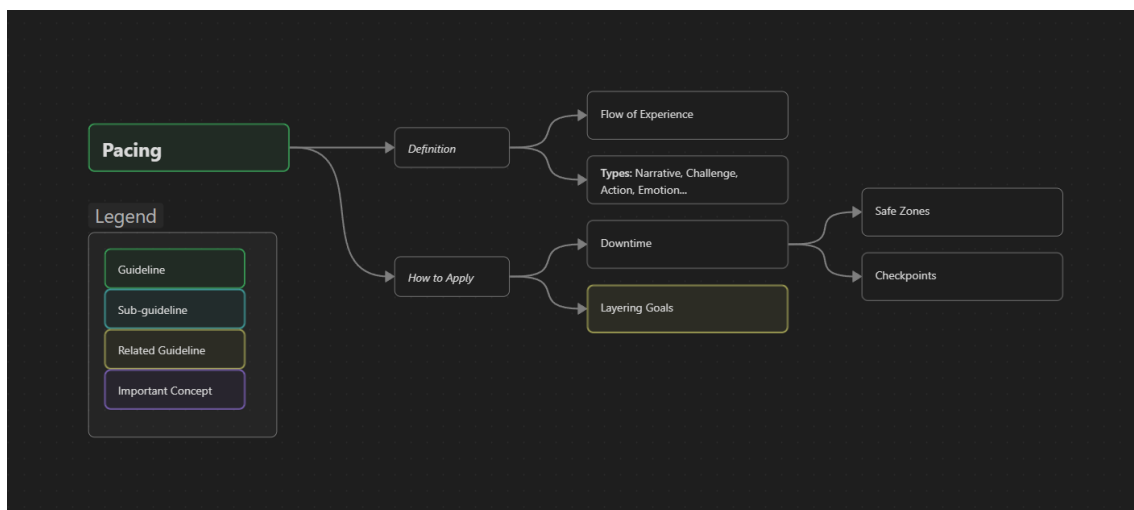


Figure A.9: Diagram that displays an overview of the structure of the **Pacing** guideline.

A.1.2 Guidelines: Guidance

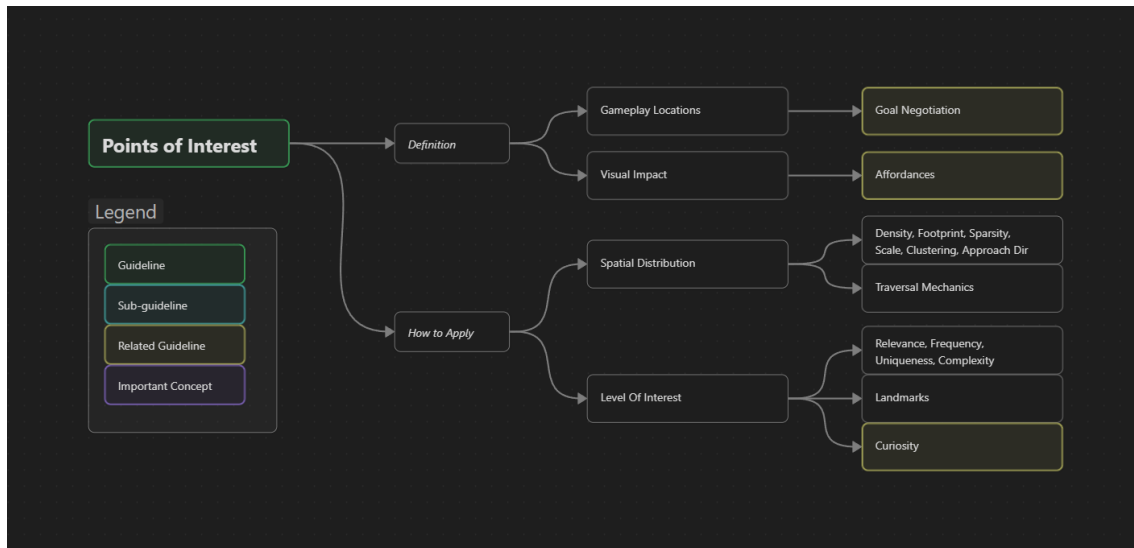


Figure A.10: Diagram that displays an overview of the structure of the **POIs** guideline.

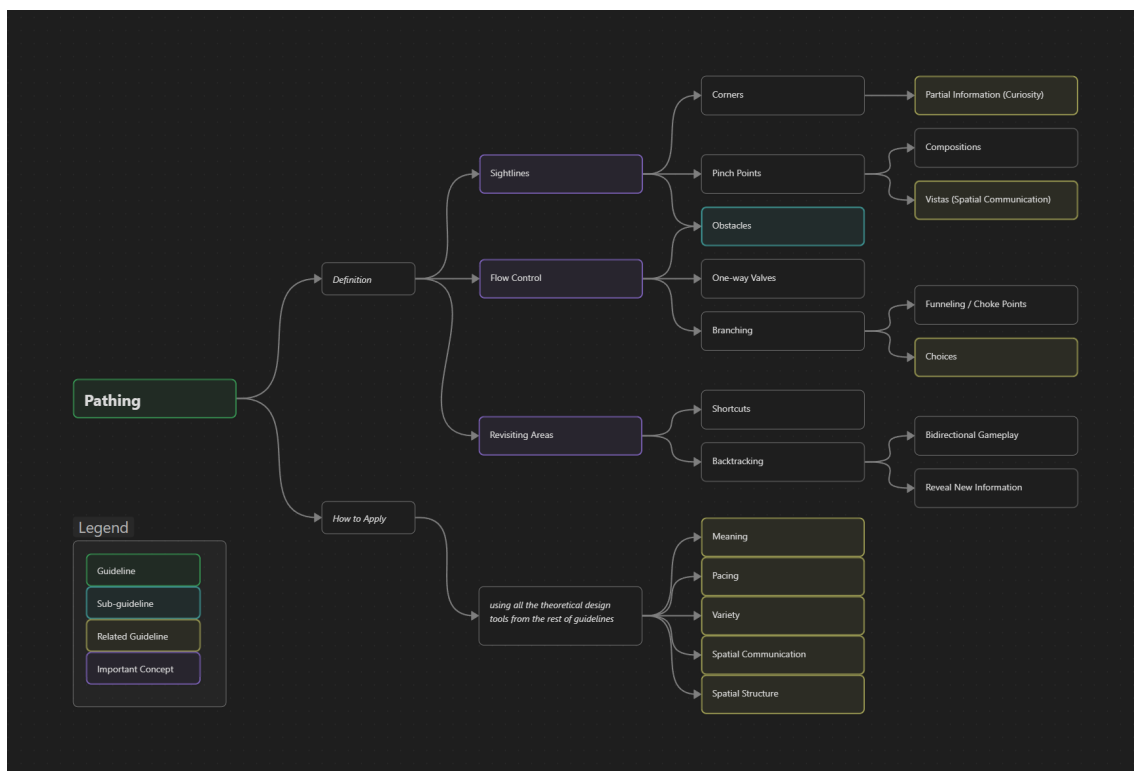


Figure A.11: Diagram that displays an overview of the structure of the **Pathing** guideline.

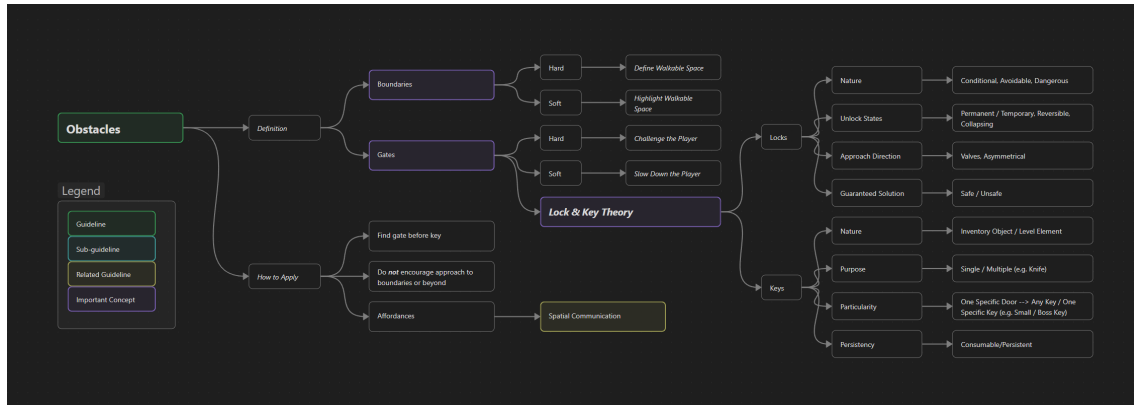


Figure A.12: Diagram that displays an overview of the structure of the **Obstacles** guideline.

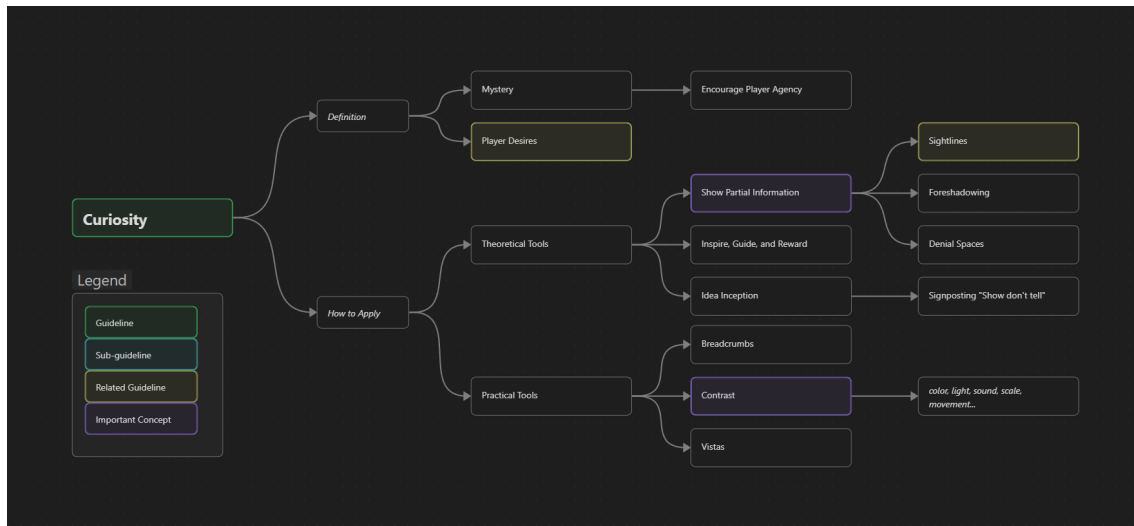


Figure A.13: Diagram that displays an overview of the structure of the **Curiosity** guideline.

A.1.3 Guidelines: Spaces

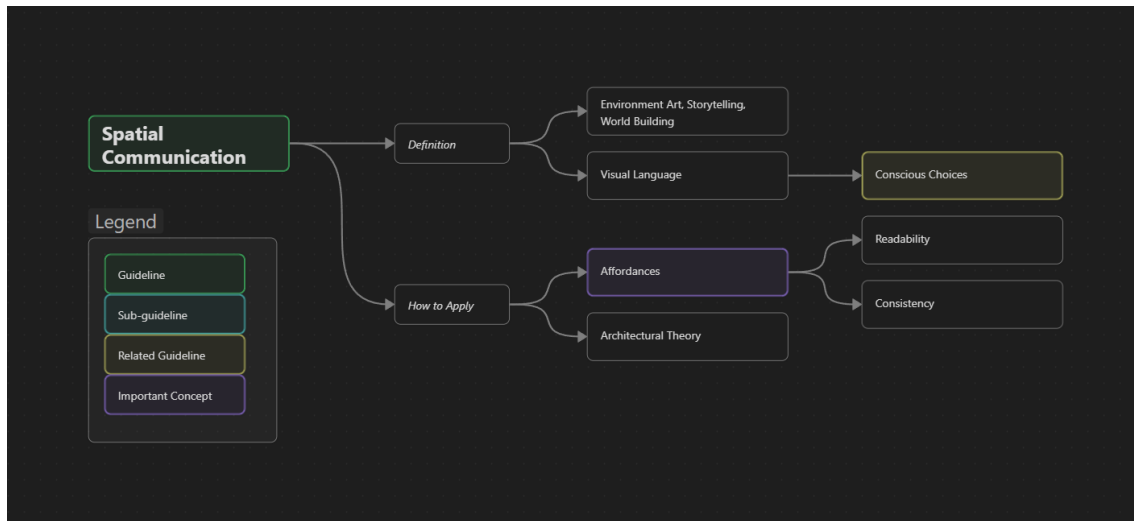


Figure A.14: Diagram that displays an overview of the structure of the **Spatial Communication** guideline.



Figure A.15: Diagram that displays an overview of the structure of the **Spatial Structure** guideline.

B

Example Vistas from the Second Artefact

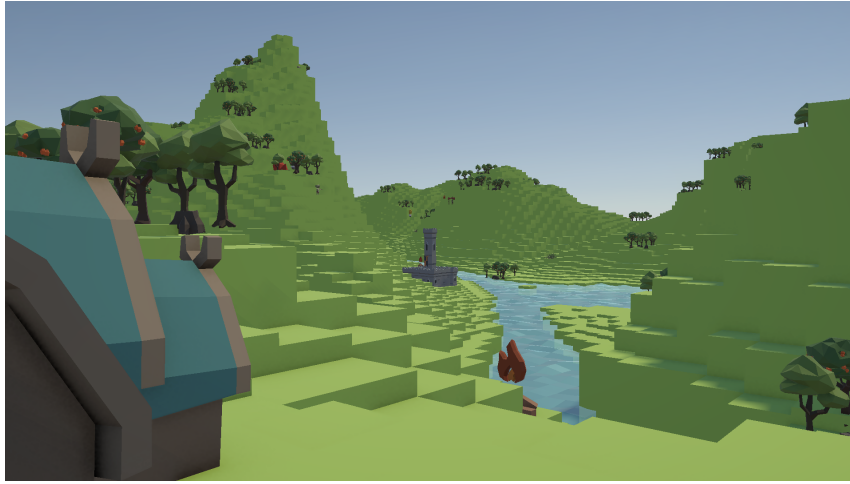


Figure B.1: Vista from the player spawn where the main goal (castle) is visible along the riverbed.

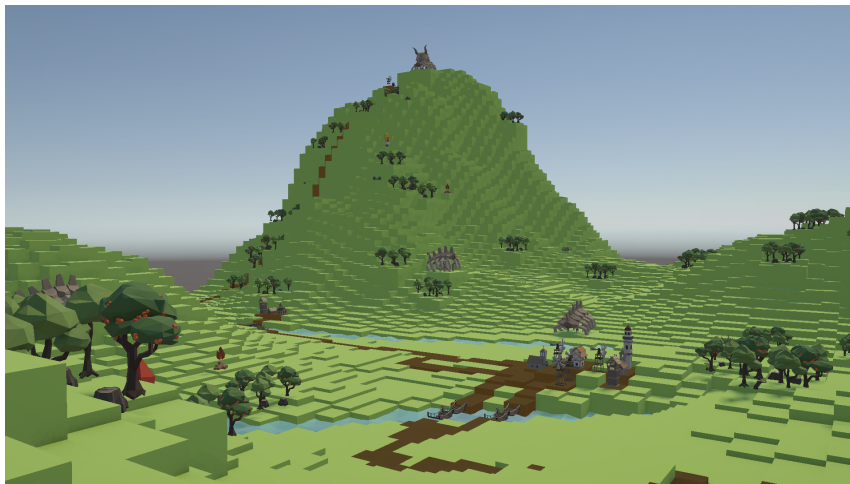


Figure B.2: Vistas from a path that leads to a village in a valley, with a foreboding enemy camp at the top of a mountain.



Figure B.3: Village from which the main goal (castle) is visible at the top of a mountain.

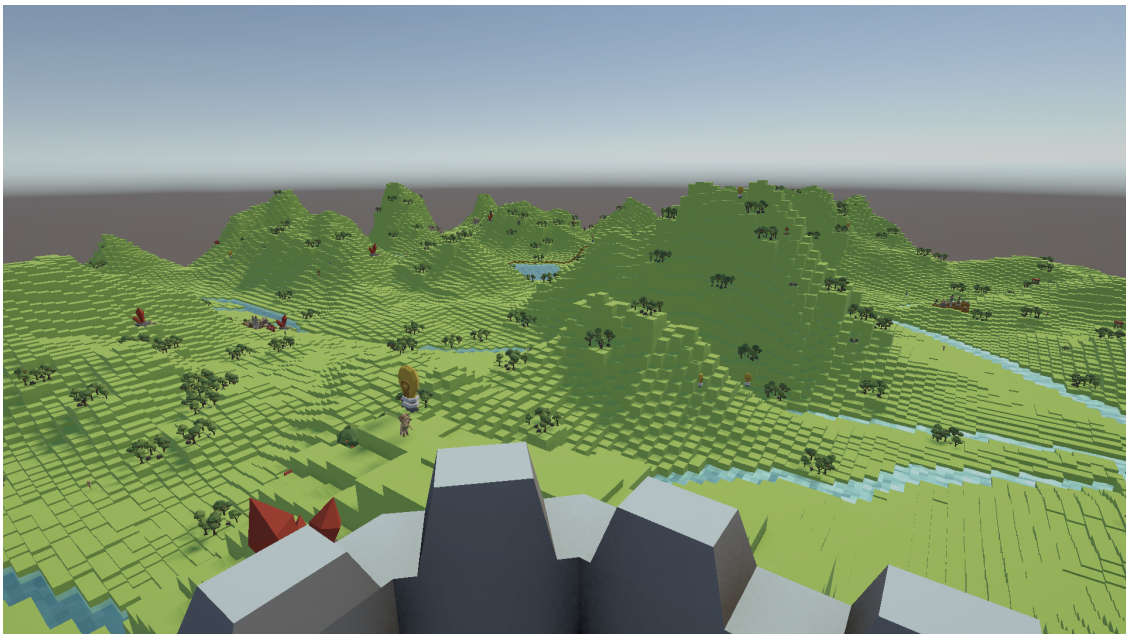


Figure B.4: Same generation as last figure, from the point of view of the castle. Here two towns can be located in the distance.

C

Level Design Sources Outline

This chapter outlines the main sources used to extract all the knowledge about level design in the research phase of this work. This is necessary due to how many of the sources overlap in the topics they cover and how each individual source often addresses multiple concepts. Including these reference would not only be an arduous task, but it would also clutter all the text and result confusing to the reader, instead of providing useful information about the origin of the different concepts and statements. For these reasons this approach serve as an alternative. What follows is a list of sources about level design and its integration with PCG, where each entry summarizes the concepts it contributes to.

Publications

- **A Phenomenological Approach to the Design of Spatial Experience in Open-world Games.** [50] Research article that explores how players perceive and make sense of the world around them. This source explains the difference between a space and a place. It is used as a foundation to define the meaning guideline, and it also collects several concepts about hidden guidance and orientation.
- **Exploration in Open-World Video games: Environment, Items, Locations, Quests, and Combat in The Witcher 3.** [4] Research article that explores many concepts of open-world games pertinent to level design. These include unpredictability, immersion, curiosity, POIs, goals, and more. It defines exploration as the process of locating POIs.
- **Level Design Patterns in 2D Games.** [27] Paper that provides a list of many key concepts proper of 2D level design, which can be also applied to 3D spaces. These include guidance, breadcrumbs, safe zones, pacing, foreshadowing, curiosity, layering, branching, choices, risk VS reward, and more.

Book Chapters

- **The Art of Game Design: A Book of Lenses.** [11] This book is used as the main source to explain how level design can influence a game experience. It explains how to design with a focus on the intended player experience, and explores goals, motivation, meaningful choices, challenge, feedback, pacing, and more.

- **Level Up! The guide to great video game design: Levels.** [25] This source merges level design principles, with practices and game design considerations all in one. It explains types of spaces and their characteristics, how to approach level prototyping, it talks about different types of variety to keep the experience fresh, it mentions the importance of goals, and it also includes a section to talk about the integration of PCG.
- **An Architectural Approach to Level Design.** [8] This book explains most of the concepts explored in the spatial communication and spatial structure guidelines. It mentions layout hierarchies, wayfinding techniques (landmarks, sightlines, etc.), it talks about spatial readability and affordances, pacing through spaces, and the experiential impact different structures can have over the player.
- **Level Design Practices for Independent Games.** [44] This source is the main reference that explains how level design can be defined as the process that implements game design, and how it should be structured around the game's core mechanics. It focuses on practices for a development process, like the implementation of iteration, or the use of an object-oriented approach. However, it gets diluted in level design patterns and game design considerations, such as pacing, challenge balancing, guidance, sandbox spaces, and emergent gameplay.
- **The Illusion of Choice - How to hide linearity of levels.** [51] This source explicitly states the concept of 'hidden guidance'. It explains several guidance concepts like landmarks, branching, types of narrative deterrents, and more. It also explains the consequences of an unsuccessful implementation, like the lack of engagement or immersion of the player.
- **Level Design Planning for Open-World Games.** [23] This book chapter explains a useful practice of creating a living documentation. It explains how to define the world map and the list of important locations, along with all their metrics and properties. It also explains a possible workflow, from pitching the idea and greyboxing, to art and polish. However, it also explains how a toolkit for level design cannot be provided, since 'any game's needs are different from one another'. This quote was important to define the scope and purpose of the guidelines.
- **Procedural Generation in Game Design: Meaning.** [47] This source is used as a foundation of the meaning guideline. It explains how meaning is born from connections, and the different types of connections there are. Although this source sometimes confuses the concept of meaning with importance or relevance, it explores how this meaning can exist or not in procedurally generated games.
- **Procedural Generation in Game Design: Graphs and Cycles.** [52] This source explains how to understand game progression from a mathematical perspective using graph theory. It explores many concepts for graph traversal that implement tools presented in the pathing guideline, such as branching,

shortcuts, one-way valves, critical paths, and more. It also serves as the main source to define gates, through the theory of types of keys and locks mentioned in the obstacles guideline.

Articles and Web Sources

- **My Level Design Guidelines (M. Barclay).** [22] Web article that presents several key concepts of level design from an industry professional. It explains concepts from several guidelines all at once, like themes, pacing, signposting, landmarks, denial spaces, POIs, breadcrumbs, spatial lures, boundaries, vistas, visual languages, set pieces, gates, objectives (goals), rewards, branching, player agency, sandbox levels, choke points, and one-way valves.
- **Level Design Fundamentals.** [53] Article published by Unreal Engine that explores a wide variety of concepts in level design. These include core mechanics, possibility space, shape language, space theory (negative space, occlusion, and player perception), metrics, scale, cognitive maps, graph theory, hierarchies, player motivation, rewards, and player choice.
- **The Level Design Book.** [54] Digital book which serves as a guide that explains how to create a game from the ground up. It presents a workflow pipeline, explaining pre-production, game design, layouts, scripting, environment art, and release, among others. It explains important theory about the use of lighting as a spatial lure along with framing theory. It explains layouts and greyboxing, and how it can be integrated in an industry pipeline. It also provides dozens of tools and resources to start practising game development.
- **‘Define first, design levels later’ (Translated from Spanish).** [24] Industry professional shares his insights about game design documents and paper prototyping in this video. He explains different tools on how to properly use drawing tools to communicate ideas on paper. Other entries of his channel explore block outs, understanding levels as game experiences, types of spaces, and open-world guidance, among others.

Talks and Developer Insights

- **Spatial Communication in Level Design.** [5] Talk at Develop Digital (2020) that explores many concepts in pathing, curiosity, goal, and spatial communication. It provides a practical example on how the following concepts apply in a space: greyboxing, clear goals, sightlines, cognitive maps, reinforcing goals, affordances, bidirectional gameplay, contrast and spatial lures, denial spaces, one-way valves, vistas/vantage points, branching, mystery, shortcuts, backtracking, pinch points, vocabulary, affordances, locks and keys, and design constraints.
- **Ten Principles for Good Level Design.** [45] GDC talk that explores several principles of level design. These include: visual languages, space navigation, environmental storytelling, clear objectives, layering goals, expectations, consequences, difficulty, risk vs reward, modularity, bidirectional gameplay, architectural theory, and MDA.

- **An Approach to Holistic Level Design.** [43] GDC talk that explain how a game consists of a story, gameplay, and their presentation, and how designers control these through intentionality, affordances, world building, goals, desires inception, and more.
- **Designing Radically Non-Linear Single Player Levels.** [55] This GDC talk explains how to implement branching and choices, managing pacing and player tension. It explains different layouts of spaces with high and low risk, and the different types of level elements that are used to create these, such as attractors or deterrents. This same concept is used in one of the prototypes resulting from this thesis.
- **Stop Getting Lost: Make Cognitive Maps, Not Levels.** [56] This GDC talk explains in detail how players' cognitive maps work and how they follow graph theory, where nodes, edges, and hierarchies are the basics. It also explains landmarks, minimaps drawbacks, districts, and how to use all these elements to orient your players or to get them lost.
- **Playgrounds and Level Design.** [57] Another GDC talk that explains the nature of playgrounds and sandbox spaces, and matches them to graph theory, following nodes, edges, and hierarchies. It also explains the usage of modules, landmarks, goals, obstacles, and rewards.
- **The Making of The Legend of Zelda: Breath of the Wild.** [58] This is an official video entry from Nintendo explaining different considerations behind the creation of *Zelda BOTW (2017)*. This thesis uses it as a resource to explain how modularity can be used as a tool to implement with PCG.
- **Skyrim's Modular Level Design.** [7] This GDC talk explains how games like *Skyrim (2011)* use a modular approach to create content like its dungeons. In particular, this resource is utilised in this thesis to explain how level design has an interdisciplinary nature, and how level designers need to keep active communication with the rest of the team (level art, narrative, tech team, and game direction).
- **Gyms, Zoos, and Museums: Your documentation should be in-game.** [59] This talk explains different types of in-game documentation and their purpose. This is a useful practice for level design to keep all the building blocks organised. It can also be extended to organise action blocks.
- **Sparking Curiosity-Driven Exploration Through Narrative in 'Outer Wilds'.** [60] This GDC talk serves as the main source for the curiosity guideline. It explains how to inspire curiosity by allowing the players to pull information, instead of pushing it onto them. It then explains how to guide the player along these attractors maintain curiosity and push the flow forward. It also talks about layering goals and rewards.

- **Designing for Curiosity in Outer Wilds.** [61] This talk given in the Full Indie Summit provides a list of concepts used to make the player curious. It particularly focus on how *Outer Wilds (2019)* frames it, but its information can be extrapolated to other games. It mentions concepts like: layering goals, pacing, clear objectives, meaningful choices, meaning (narrative relatedness to the player), and signposting.